

PIMPRI CHINCHWAD EDUCATION TRUST'S
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
SECTOR NO. 26, PRADHIKARAN, NIGDI, PUNE - 411044



HONOR MAJOR PROJECT

Project Group ID: 1

Title:

VulnShield-DNN: TD3-Based Fault-Sensitive Channel Identification, Cross-Layer Interaction Analysis, and Budget-Aware Robustness Enhancement for Deep Neural Networks

Project Group Members:

PRN No.	Name of Student
123B1D048	Ishwar Sonawane
123B1D056	Adhya Bhagat
123B1D062	Nihar Salvi

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

ACADEMIC YEAR 2026 – 2027

Project Guide

Dr. Mobin Tamboli

Head of the Department

Dr. Govinda Sambare

TABLE OF CONTENTS

TABLE OF CONTENTS

ABSTRACT	i
1. INTRODUCTION	1
1.1 Background and Context	1
1.2 Deep Neural Network Reliability	2
1.3 Hardware Faults in Deep Neural Networks	2
1.4 Research Motivation	3
1.5 Problem Overview	3
1.6 Contributions of the Proposed Work	4
2. PROBLEM FORMULATION AND RESEARCH OBJECTIVES	5
2.1 Problem Statement	5
2.2 Research Challenges	6
2.3 Research Questions	6
2.4 Motivation and Need of the Project	7
2.5 Objectives	8
2.6 Scope of the Proposed Work	9
3. LITERATURE SURVEY AND RESEARCH GAP	10
3.1 Introduction to DNN Reliability Research	10
3.2 Software-Based Fault Injection Techniques	11
3.3 DNN Vulnerability Assessment Methods	12
3.4 Reinforcement Learning for DNN Vulnerability Detection	13
3.5 Fault-Aware Training and DNN Protection	14
3.6 Simultaneous and Cross-Layer Fault Analysis	15
3.7 Adversarial Sensitivity and Fault Vulnerability	16
3.8 Comparative Analysis of Existing Approaches	17
3.9 Research Gap Analysis	19
3.10 Novelty and Research Contributions	20
4. PROPOSED VULNSHIELD-DNN FRAMEWORK	22
4.1 Overview of the Proposed Framework	22
4.2 Design Principles	23
4.3 Overall System Architecture	24
4.4 Functional Block Diagram	25
4.5 End-to-End Framework Workflow	27
4.6 Fault Injection Module	28
4.7 Channel Vulnerability Representation	29
4.8 TD3-Based Vulnerability Detection	30
4.9 Vulnerability Ranking and Selection	32
4.10 Cross-Layer Interaction Analysis	33
4.11 Adversarial-Sensitivity Comparison	34
4.12 Vulnerability-Aware Fault Fine-Tuning	35
4.13 Budget-Aware Protection Strategy	36

5. REINFORCEMENT LEARNING-BASED VULNERABILITY DETECTION	38
5.1 Reinforcement Learning Formulation	38
5.2 Environment Design	39
5.3 State Representation	40
5.4 Action Space	40
5.5 Reward Function	41
5.6 TD3 Agent Architecture	42
5.7 Vulnerability Search under Limited Injection Budget	43
5.8 Vulnerability Ranking Generation	44
6. FAULT-AWARE PROTECTION AND CROSS-LAYER ANALYSIS	45
6.1 Single-Channel Fault Analysis	45
6.2 Cross-Layer Fault Interaction	46
6.3 Fault-Sensitive Channel Selection	47
6.4 Vulnerability-Aware Fine-Tuning	48
6.5 Combined Clean and Faulted Training Objective	49
6.6 Protection Budget Formulation	50
6.7 Protected DNN Generation	51
7. EXPERIMENTAL METHODOLOGY	52
7.1 Experimental Setup	52
7.2 Dataset	53
7.3 DNN Architectures	53
7.4 Fault Model	54
7.5 Fault Injection Procedure	55
7.6 Baseline Vulnerability Detection Methods	56
7.7 TD3 Experimental Configuration	57
7.8 Adversarial Testing using FGSM and PGD	58
7.9 Protection Budgets	59
7.10 Evaluation Metrics	60
7.11 Validation Strategy	61
8. EXPECTED RESULTS AND EVALUATION	62
8.1 Vulnerability Detection Performance	62
8.2 Search Efficiency under Limited Fault Budget	63
8.3 Comparison with Random and Heuristic Methods	64
8.4 Comparison with DDPG-Based Search	65
8.5 Cross-Layer Fault Interaction Results	66
8.6 Fault versus Adversarial Sensitivity	67
8.7 Protection Performance under Different Budgets	68
8.8 Robustness and Clean-Accuracy Trade-off	69
8.9 Computational Overhead	70
9. DISCUSSION	71
9.1 Interpretation of Vulnerability Patterns	71
9.2 Effectiveness of TD3-Based Search	72
9.3 Significance of Cross-Layer Interactions	73
9.4 Effectiveness of Ranking-Guided Protection	74
9.5 Limitations of the Proposed Framework	75
9.6 Threats to Validity	76

10. EXPECTED CONCLUSION AND FUTURE WORK	77
10.1 Expected Conclusion	77
10.2 Research Contributions	78
10.3 Future Extensions	79
REFERENCES	80
APPENDIX	84
A. Experimental Configuration	84
B. Hyperparameter Settings	85
C. Additional Experimental Results	86

ABSTRACT

Deep Neural Networks (DNNs) are increasingly deployed in safety-critical and edge computing systems where prediction reliability is an important requirement. Hardware-related faults, including transient faults and other computational errors, can corrupt model parameters or intermediate activations and may result in incorrect predictions. The impact of such faults, however, is not uniform across the network. While faults affecting many internal channels may have limited influence on model performance, faults affecting a small number of fault-sensitive channels can cause significant accuracy degradation or misclassification.

Existing DNN fault-vulnerability assessment approaches commonly rely on extensive fault-injection experiments, predefined activation- or gradient-based heuristics, or independent layer-wise analysis. These approaches can require substantial computational effort and may not adequately identify interactions between vulnerable components across different network layers. Furthermore, vulnerability assessment is not always directly connected to a targeted model-protection strategy, particularly when only a limited computational or protection budget is available.

This project proposes VulnShield-DNN, a software-based framework for identifying and mitigating fault-sensitive channels in deep neural networks. The framework uses controlled software fault injection to evaluate the effect of channel-level faults on DNN predictions. A Twin Delayed Deep Deterministic Policy Gradient (TD3) agent is employed to intelligently select fault-injection locations and identify high-impact channels under a limited fault-injection budget. The resulting vulnerability ranking is subsequently used to investigate selected cross-layer fault interactions and to guide targeted fault-aware fine-tuning.

The proposed framework will compare TD3-based vulnerability identification with random search, activation-based ranking, gradient or Taylor-based ranking, and layer-wise Deep Deterministic Policy Gradient (DDPG). Fault-aware fine-tuning will expose selected vulnerable channel outputs to simulated faults during training while jointly preserving clean-data performance. The framework will be evaluated using convolutional neural networks, primarily ResNet-18 and additionally VGG-16, on the CIFAR-10 dataset. Evaluation will consider faulted accuracy, clean accuracy, vulnerability-discovery efficiency, robustness improvement, and computational overhead under different protection budgets.

The proposed work aims to establish whether reinforcement-learning-guided channel identification can provide a more efficient basis for targeted DNN reliability enhancement than conventional vulnerability-ranking approaches, while also examining the role of cross-layer fault interactions in model degradation.

1. INTRODUCTION

Deep Neural Networks (DNNs) have become a fundamental component of modern artificial intelligence systems and are increasingly deployed in applications that require reliable and accurate decision-making. Convolutional Neural Networks (CNNs), in particular, are widely used for computer vision and embedded intelligence applications. Their deployment has expanded from conventional computing environments to edge devices and safety-sensitive systems, where model reliability is influenced not only by the neural network architecture and training process but also by the reliability of the underlying computing hardware.

During DNN inference, hardware and computational faults can affect model parameters, intermediate activations, or arithmetic operations. Such faults may arise from transient phenomena, memory corruption, voltage variations, timing violations, hardware aging, or other sources of computational errors. In safety-critical and resource-constrained environments, even an isolated internal fault can potentially propagate through subsequent layers and alter the final prediction. The resulting error may remain undetected because the network can continue its inference process while producing an incorrect output.

The effect of a fault is not uniform across the internal components of a DNN. Different neurons, filters, and feature channels contribute differently to the computation performed by subsequent layers. Consequently, corrupting one channel may produce little or no measurable change in the final prediction, whereas corruption of another channel may cause a substantial increase in prediction loss, accuracy degradation, or misclassification. This non-uniformity creates an important reliability-analysis problem: identifying the internal components whose corruption has the greatest impact on model behavior.

Software-based fault injection provides a practical mechanism for studying this problem without requiring physical fault-injection hardware. Controlled perturbations can be introduced into DNN computations and the resulting changes in model behavior can be measured systematically. Existing fault-injection and vulnerability-assessment approaches have demonstrated the feasibility of evaluating DNN resilience through software simulation. However, exhaustive fault injection can become computationally expensive as the number of layers, channels, and possible fault locations increases.

To reduce this search complexity, analytical and learning-based approaches have been investigated for identifying vulnerable components. Activation-based and gradient-based methods can provide information about potentially important components, while reinforcement-learning-based approaches can adaptively guide fault-injection experiments toward potentially high-impact locations. Nevertheless, vulnerability assessment remains challenging when the analysis is performed independently across layers, because faults affecting components in different parts of a network may interact and produce degradation that cannot be inferred from their individual effects alone.

Another important challenge is the transition from vulnerability assessment to model protection. Identifying fault-sensitive components is useful only if the resulting information can be translated into an effective reliability-improvement strategy. Protecting every internal channel uniformly may introduce unnecessary computational and training costs, particularly when only a limited portion of the model can be subjected to additional robustness training. Therefore, the ability to prioritize the most consequential channels under a defined protection budget is an important consideration for practical DNN reliability enhancement.

These challenges motivate research into efficient, targeted, and experimentally validated approaches for DNN fault vulnerability analysis and robustness enhancement. The proposed VulnShield-DNN framework investigates this problem at the channel level by combining reinforcement-learning-guided vulnerability identification, selected cross-layer interaction analysis, and targeted fault-aware fine-tuning. The framework is designed to determine whether intelligently selected fault-injection experiments can identify high-impact channels more efficiently than conventional search and ranking approaches, and whether the resulting vulnerability information can be used to improve DNN resilience under limited protection budgets.

The study therefore focuses on the relationship between fault location, vulnerability, cross-layer interaction, and targeted protection. By evaluating these aspects within a common experimental framework, the work aims to provide a systematic approach for understanding and improving the reliability of deep neural networks subjected to simulated internal faults.

1.1 Background and Context

Deep Neural Networks (DNNs) have become a core technology for modern artificial intelligence and are widely used for tasks such as image classification, object recognition, autonomous decision-making, and intelligent control. The increasing computational capability of modern hardware has enabled these models to be deployed beyond conventional computing platforms, including edge and embedded systems where inference is performed under constrained computational and energy resources.

As DNNs are increasingly incorporated into applications where reliable predictions are important, the reliability of their computational execution becomes an important consideration. A trained neural network does not operate in isolation; its inference process depends on the correct execution of numerical operations, memory accesses, parameter storage, and intermediate computations. Consequently, errors occurring within the underlying computational process can affect the output even when the trained model itself has not been intentionally modified.

Hardware-related faults represent one such source of reliability degradation. Transient faults, memory-related errors, voltage fluctuations, timing violations, and hardware aging can alter parameters or intermediate activations during neural-network execution. Depending on where such an error occurs, its effect on the final prediction may range from negligible to substantial. This makes DNN reliability different from simply evaluating the accuracy of a model under normal operating conditions.

An important characteristic of DNNs is that their internal components do not contribute equally to the final prediction. Different layers, neurons, filters, and feature channels can have different levels of influence on the network's computational behavior. Therefore, the same type of fault introduced at different internal locations can produce significantly different effects on model performance. Understanding this variation is essential for determining which parts of a DNN require greater attention from a reliability perspective.

Traditional reliability assessment can involve testing a large number of possible fault locations to determine their effects. However, the number of potential locations increases substantially with network depth and the number of internal channels. This creates a practical challenge when reliability analysis must be performed within limited computational resources.

These characteristics motivate the study of fault vulnerability at the internal-channel level of DNNs. Rather than treating the network as a uniformly vulnerable structure, the research focuses on understanding how faults affecting different internal components influence model behavior and how this vulnerability information can subsequently support targeted reliability enhancement.

This forms the foundational context for VulnShield-DNN, which investigates systematic identification and analysis of fault-sensitive channels in deep neural networks and their use in targeted robustness enhancement.

1.2 Deep Neural Network Reliability

Deep Neural Network reliability refers to the ability of a trained neural network to maintain its expected computational and predictive behavior when subjected to disturbances or faults during operation. While conventional DNN evaluation primarily focuses on metrics such as classification accuracy under fault-free conditions, reliability analysis additionally considers how the model behaves when its computations, parameters, or intermediate representations are perturbed.

A DNN consists of multiple computational layers that transform input data through a sequence of learned operations. During inference, each layer generates intermediate representations that are subsequently processed by later layers. An error introduced at an intermediate stage can therefore propagate through the remaining computation and potentially influence the final prediction. The extent of this propagation depends on the location and characteristics of the affected component.

The reliability of a DNN is consequently influenced by the sensitivity of its internal computational components. Channels, filters, neurons, and other intermediate structures may have different contributions to the final output. A perturbation affecting one component may be absorbed by subsequent computations, whereas a similar perturbation affecting another component may substantially alter the network's output. This non-uniform behavior makes component-level reliability assessment important for understanding the actual fault sensitivity of a DNN.

Reliability analysis is particularly important when DNNs are deployed in environments where computational errors can have significant consequences. In such settings, evaluating only the clean-data performance of a model is insufficient to characterize its operational reliability. The model must also be assessed according to its ability to tolerate faults without unacceptable degradation in its predictions.

One approach to studying DNN reliability is to deliberately introduce controlled faults into the computational process and observe the resulting change in model behavior. Such controlled fault analysis makes it possible to measure the sensitivity of different internal components and identify locations where faults have a greater impact on model performance.

However, comprehensive reliability assessment can become computationally expensive because modern DNNs may contain a large number of layers and internal channels. Evaluating every possible fault location individually can require a substantial number of experiments. This creates a need for efficient methods that can prioritize the most consequential components for detailed analysis.

In the context of this research, DNN reliability is therefore considered from the perspective of fault sensitivity, vulnerability identification, and targeted robustness improvement. The objective is not simply to determine whether a model can tolerate faults, but to understand which internal components are most sensitive to faults and how this information can be used to improve the model's reliability under constrained protection resources.

1.3 Hardware Faults in Deep Neural Networks

Hardware faults are unintended errors that occur during the execution of computational operations or the storage and transfer of data within a computing system. In DNN inference, such faults can

affect model parameters, intermediate activations, memory contents, or numerical computations and consequently alter the values propagated through the network.

Hardware-related faults can arise from several physical and operational conditions. Transient faults, such as soft errors caused by environmental or electrical disturbances, may temporarily alter a stored or computed value. Memory-related faults can corrupt parameters or intermediate activation values stored during inference. Voltage fluctuations and timing violations can cause computational operations to produce incorrect results, while hardware aging can progressively affect the reliability of computational components. The effect of these faults depends on both the nature of the fault and the location at which it occurs.

In a DNN, a fault introduced into an intermediate activation can propagate through subsequent layers. If the affected value contributes strongly to later computations, the resulting error may become amplified and influence the final prediction. Conversely, a fault occurring in a less influential component may have little observable effect because subsequent computations can reduce or compensate for the disturbance.

This makes fault location an important factor in DNN reliability analysis. Two faults with similar characteristics can produce substantially different outcomes when introduced at different channels or layers. Therefore, assessing DNN fault tolerance requires more than measuring the model's overall response to an arbitrary fault; it requires identifying the internal locations that are particularly sensitive to computational corruption.

Software-based fault injection provides a controlled approach for studying these effects. Instead of requiring physical modification of the underlying hardware, faults can be simulated at selected points in the neural-network computation. The resulting changes in prediction accuracy, loss, or other model-performance measures can then be used to characterize the vulnerability of the affected components.

However, the increasing depth and width of modern DNN architectures create a large fault-injection search space. A systematic assessment of every possible channel and fault condition can therefore require substantial computational resources. This motivates efficient methods for prioritizing fault-sensitive components and analyzing their contribution to overall model degradation.

Within this research, hardware faults are therefore represented through controlled software fault injection to study the vulnerability of internal DNN channels. The analysis focuses on determining how faults at different locations affect model behavior and how the resulting vulnerability information can support selective rather than uniform reliability enhancement.

1.4 Research Motivation

The increasing deployment of Deep Neural Networks in computing environments where reliable operation is required creates a need to understand their behavior under hardware-induced faults. Conventional model evaluation primarily measures performance under fault-free conditions and therefore does not fully characterize how a trained network responds when internal computations are corrupted during inference.

A major motivation for this research is the non-uniform distribution of fault sensitivity within a DNN. Different internal channels can contribute differently to the final prediction, meaning that the effect of an injected fault may vary considerably depending on its location. Identifying these fault-sensitive channels can therefore provide more meaningful reliability information than treating all internal components equally.

Another motivation is the computational cost of vulnerability assessment. Modern DNNs can contain a large number of channels across multiple layers. Exhaustively injecting faults into every possible location can require a substantial number of evaluations. An efficient method for directing fault-injection experiments toward potentially high-impact locations can reduce unnecessary evaluations while preserving the ability to identify significant vulnerabilities.

The research is also motivated by the possibility of interactions between faults occurring at different layers. Independent channel-level analysis may not adequately characterize situations in which multiple faults collectively produce a different effect from their individual impacts. Examining selected cross-layer combinations can provide additional insight into how vulnerabilities propagate through the network.

Finally, vulnerability identification is most useful when it can support a practical reliability-improvement strategy. Applying additional protection uniformly to every channel may require unnecessary computational and training resources. A vulnerability-guided approach can instead prioritize components according to their measured fault sensitivity and apply protection within a defined resource budget.

These considerations motivate the development and evaluation of VulnShield-DNN, with a focus on efficient fault-sensitive channel identification, cross-layer vulnerability analysis, and targeted robustness enhancement for DNNs.

1.5 Problem Overview

The reliability of a Deep Neural Network under hardware-induced faults depends not only on the presence of a fault but also on where within the network the fault occurs. Internal channels across different layers can exhibit substantially different levels of sensitivity to corruption. Consequently, a DNN may contain a relatively small subset of channels whose failure has a disproportionately large effect on prediction performance.

Identifying these fault-sensitive channels presents a significant search problem. A straightforward approach would require fault injection across a large number of channels and repeated evaluation of the resulting model behavior. As the size and depth of a DNN increase, the number of potential fault locations also increases, making exhaustive vulnerability assessment increasingly expensive.

The problem is further complicated by the fact that the effect of multiple faults may not be equivalent to the independent effects of those faults. Channels located in different layers are connected through the network's forward computation, and simultaneous faults may therefore interact as their effects propagate through subsequent layers. An analysis limited to individual channels may fail to capture such cross-layer effects.

Another consideration is the practical use of the identified vulnerabilities. If all channels are subjected to additional robustness training, the computational and training cost may become unnecessarily high. Conversely, protecting only a limited number of channels requires a reliable mechanism for determining which channels should receive priority.

Thus, the problem addressed by this research consists of three connected aspects:

1. Efficiently identifying fault-sensitive channels without requiring exhaustive fault injection across the complete DNN.
2. Analyzing selected cross-layer fault interactions to determine whether combined faults exhibit behavior that cannot be inferred from individual channel vulnerabilities alone.

3. Using the identified vulnerability information for targeted robustness enhancement under a limited protection budget.

VulnShield-DNN addresses these aspects as a unified research problem by investigating reinforcement-learning-guided vulnerability identification, cross-layer fault analysis, and targeted fault-aware robustness enhancement. The effectiveness of these components will be evaluated through controlled software fault-injection experiments and comparison with established vulnerability-identification approaches.

1.6 Contributions of the Proposed Work

The proposed VulnShield-DNN framework investigates a targeted approach for analyzing and improving the fault resilience of Deep Neural Networks. The principal contributions of the proposed work are:

1. TD3-based fault-sensitive channel identification:
A reinforcement-learning-based approach is proposed to guide software fault-injection experiments toward potentially high-impact internal channels, with the objective of reducing unnecessary exploration of low-impact fault locations.
2. Channel-level vulnerability ranking:
The framework generates a vulnerability ranking based on the observed impact of injected faults, enabling the identification and prioritization of channels that exhibit greater sensitivity to fault-induced degradation.
3. Cross-layer fault interaction analysis:
Selected vulnerable channels from different layers are analyzed in combination to investigate whether simultaneous faults can produce interaction effects that differ from the effects observed during independent channel-level fault injection.
4. Vulnerability-guided robustness enhancement:
The identified fault-sensitive channels are used to guide targeted fault-aware fine-tuning rather than applying additional protection uniformly across the entire network.
5. Budget-aware protection:
The proposed approach considers limited fault-injection and protection resources, enabling the investigation of how different protection budgets affect vulnerability identification and DNN robustness.
6. Comparative evaluation of vulnerability-identification approaches:
The proposed TD3-based approach will be evaluated against random fault selection, activation-based ranking, gradient/Taylor-based ranking, and DDPG-based search to assess differences in vulnerability discovery and computational efficiency.
7. Joint evaluation of reliability and model performance:
The framework evaluates both faulted-model robustness and clean-data performance, allowing the effectiveness of targeted protection to be assessed without considering fault tolerance in isolation.

Overall, the proposed work aims to establish a systematic connection between fault injection, vulnerability identification, cross-layer analysis, and targeted robustness enhancement, providing a basis for evaluating resource-efficient reliability improvement in deep neural networks.

2. PROBLEM FORMULATION AND RESEARCH OBJECTIVES

2.1 Problem Statement

Deep Neural Networks can exhibit different levels of sensitivity to hardware-induced faults depending on the internal location affected. Although software-based fault injection can be used to evaluate this sensitivity, identifying the most consequential channels across a DNN can require a large number of fault-injection experiments. The problem becomes more challenging when the analysis must consider interactions between vulnerable channels located in different layers and when only limited computational resources are available for vulnerability assessment and subsequent protection.

Existing approaches based on random fault injection, activation statistics, gradient-based measures, and reinforcement-learning-based search provide different mechanisms for identifying potentially vulnerable components. However, an efficient framework is required to systematically connect fault-sensitive channel identification, cross-layer interaction analysis, and targeted robustness enhancement under constrained budgets.

Therefore, this research addresses the problem of efficiently identifying and prioritizing fault-sensitive channels in Deep Neural Networks, analyzing the effects of selected cross-layer fault combinations, and using the resulting vulnerability information to guide targeted fault-aware robustness enhancement within a limited protection budget.

The study investigates whether a TD3-based reinforcement-learning strategy can efficiently guide fault-injection exploration toward high-impact channels and whether the identified vulnerability ranking can provide a suitable basis for targeted robustness enhancement. The proposed approach will be evaluated through controlled software fault injection and compared with selected conventional and reinforcement-learning-based vulnerability-identification methods.

The central research problem can therefore be expressed as:

How can fault-sensitive channels in a Deep Neural Network be efficiently identified under a limited fault-injection budget, how do selected vulnerabilities interact across network layers, and how can this vulnerability information be used to achieve targeted robustness enhancement without unnecessarily increasing computational and training costs?

2.2 Research Challenges

The proposed research involves several technical challenges that must be addressed to achieve efficient and reliable fault vulnerability analysis of Deep Neural Networks.

1. Large Fault-Injection Search Space

Modern DNNs contain a large number of internal channels distributed across multiple layers. Evaluating every possible channel through repeated fault-injection experiments can require substantial computational resources. Therefore, an efficient mechanism is required to prioritize potentially high-impact fault locations while operating under a limited fault-injection budget.

2. Non-Uniform Fault Sensitivity

Different channels within the same network may respond differently to similar fault conditions. Some channels may exhibit minimal influence on the final prediction, while faults affecting other channels may cause significant degradation. Accurately distinguishing these fault-sensitive channels from relatively insensitive components is therefore a key challenge.

3. Cross-Layer Fault Interactions

The effect of simultaneous faults in channels belonging to different layers may differ from the individual effects of those faults. Since intermediate representations are propagated through subsequent layers, interactions between faults may produce compounded or non-linear effects. Capturing these interactions requires analysis beyond independent channel-level vulnerability assessment.

4. Efficient Reinforcement Learning-Based Search

Using reinforcement learning for vulnerability identification requires an appropriate formulation of the state space, action space, reward function, and exploration strategy. The RL agent must learn to identify useful fault locations while minimizing unnecessary fault-injection experiments. Maintaining an effective balance between exploration and exploitation is therefore an important challenge.

5. Vulnerability Ranking and Prioritization

Fault injection can generate vulnerability information for a large number of channels. Converting these observations into a reliable ranking is necessary for selecting the most consequential channels for further analysis and protection. The ranking mechanism must reflect actual fault impact rather than relying solely on indirect indicators such as activation magnitude or gradient values.

6. Targeted Robustness Enhancement

Identifying vulnerable channels does not by itself improve DNN reliability. The identified vulnerability information must be translated into an effective protection strategy. Applying robustness training uniformly across the entire network may introduce unnecessary computational and training costs. Therefore, the challenge is to selectively protect high-impact channels while preserving the model's clean-data performance.

7. Protection under Limited Resources

In practical deployment scenarios, computational, training, and fault-injection resources may be limited. The framework must therefore investigate how different protection budgets influence vulnerability coverage and robustness improvement, and determine whether targeted protection can provide meaningful reliability gains without requiring protection of the complete network.

8. Maintaining Clean-Model Performance

Increasing robustness to injected faults should not result in unacceptable degradation of normal model performance. The protection strategy must therefore consider the trade-off between faulted accuracy and clean accuracy when evaluating the effectiveness of targeted fault-aware fine-tuning.

9. Comparative and Experimental Validation

The effectiveness of the proposed approach must be established through controlled experiments rather than assumptions. This requires comparison with appropriate baseline methods, consistent fault models, defined evaluation metrics, and controlled protection budgets. The resulting evaluation must distinguish improvements caused by the vulnerability-guided strategy from variations arising from experimental conditions.

2.3 Research Questions

The proposed research is guided by the following research questions:

RQ1. How effectively can a TD3-based reinforcement-learning strategy identify fault-sensitive channels in a Deep Neural Network under a limited fault-injection budget?

RQ2. How does TD3-based vulnerability identification compare with random selection, activation-based ranking, gradient/Taylor-based ranking, and DDPG-based search in terms of vulnerability-discovery efficiency and computational cost?

RQ3. To what extent do simultaneous faults affecting vulnerable channels across different network layers produce interactions that differ from the effects of individual channel faults?

RQ4. Can the vulnerability ranking generated through fault injection be used to reliably prioritize channels for targeted fault-aware robustness enhancement?

RQ5. How does targeted vulnerability-aware fine-tuning affect the faulted accuracy and clean-data accuracy of the protected DNN?

RQ6. How does the available protection budget influence the trade-off between the number of protected channels, robustness improvement, clean-model performance, and computational overhead?

RQ7. Does vulnerability-guided protection provide a more resource-efficient robustness enhancement than applying fault-aware protection without vulnerability-based channel prioritization?

2.4 Motivation and Need of the Project

The increasing use of Deep Neural Networks in reliability-sensitive and resource-constrained environments creates a need for systematic methods to assess and improve their tolerance to hardware-induced faults. Conventional evaluation of DNNs generally focuses on performance under fault-free conditions and does not provide sufficient information about the internal components that are most sensitive to faults.

A major motivation for this work is the non-uniform distribution of fault vulnerability within a DNN. Since different channels contribute differently to the computation performed by subsequent layers, faults affecting different channels can produce substantially different effects on the final prediction. Identifying these fault-sensitive channels can therefore enable reliability analysis and protection to be focused on the components that have the greatest observed impact.

There is also a need to reduce the computational cost associated with fault vulnerability assessment. Exhaustive fault injection across all possible channels and fault locations can require a large number of model evaluations. An intelligent search strategy that can prioritize potentially significant fault locations may reduce unnecessary evaluations and make vulnerability assessment more practical for larger DNN architectures.

Furthermore, independent channel-level analysis may not fully represent the behavior of simultaneous faults. Since DNN layers are computationally interconnected, faults occurring at different layers may interact as their effects propagate through the network. Investigating selected cross-layer fault combinations can therefore provide additional information about the relationship between individual channel vulnerability and combined fault impact.

The project is also motivated by the need for targeted rather than uniform robustness enhancement. If only a limited number of channels can be subjected to additional fault-aware training, those channels should be selected based on evidence of their vulnerability. A vulnerability-guided protection strategy can potentially allocate limited protection resources toward the components with greater observed fault impact.

Therefore, the need for this project arises from the combination of three requirements: efficient identification of fault-sensitive DNN channels, systematic analysis of selected cross-layer fault interactions, and targeted robustness enhancement under limited protection resources. VulnShield-DNN investigates these requirements within a unified framework using reinforcement-learning-guided vulnerability identification and vulnerability-aware fault-tolerant fine-tuning.

2.5 Objectives

The primary objective of the proposed work is to develop and evaluate a systematic framework for identifying fault-sensitive channels in Deep Neural Networks and using the resulting vulnerability information for targeted robustness enhancement under constrained resources.

The specific objectives of the project are:

1. To develop a controlled software-based fault-injection methodology for evaluating the sensitivity of internal DNN channels to simulated hardware-induced faults.
2. To formulate fault vulnerability assessment as a reinforcement-learning problem and investigate the use of Twin Delayed Deep Deterministic Policy Gradient (TD3) for efficiently selecting potentially high-impact fault locations.
3. To identify and rank fault-sensitive channels based on their measured impact on DNN prediction performance.
4. To evaluate the efficiency of TD3-based vulnerability identification against selected baseline approaches, including random fault selection, activation-based ranking, gradient/Taylor-based ranking, and DDPG-based search.
5. To analyze selected simultaneous faults across different network layers and investigate whether cross-layer fault interactions produce effects that differ from those observed for individual channel faults.
6. To use the identified vulnerability ranking to prioritize channels for targeted fault-aware fine-tuning instead of applying additional robustness training uniformly across the complete network.
7. To investigate the effect of different protection budgets on the relationship between the number of protected channels, faulted-model robustness, clean-data performance, and computational overhead.
8. To evaluate whether vulnerability-guided protection can improve DNN fault tolerance while maintaining acceptable clean-model performance.
9. To establish an experimental evaluation framework that measures vulnerability-discovery efficiency, faulted accuracy, clean accuracy, robustness improvement, and computational cost under controlled fault-injection and protection conditions.

2.6 Scope of the Proposed Work

The scope of the proposed work is limited to the software-based analysis and targeted improvement of Deep Neural Network robustness against simulated hardware-induced faults. The study focuses on identifying internal channels that exhibit higher sensitivity to injected faults and investigating how this vulnerability information can be used for selective robustness enhancement.

The proposed work includes the following areas:

1. DNN Fault Vulnerability Analysis

The study will investigate the vulnerability of internal channels within selected Deep Neural Network architectures using controlled software-based fault injection. The analysis will focus on the effect of injected faults on model prediction performance.

2. Channel-Level Vulnerability Identification

The framework will operate at the internal channel level and will evaluate the relative impact of faults affecting different channels. The resulting observations will be used to generate a vulnerability ranking for prioritizing fault-sensitive channels.

3. TD3-Based Vulnerability Search

Twin Delayed Deep Deterministic Policy Gradient (TD3) will be investigated as the reinforcement-learning mechanism for guiding fault-injection experiments toward potentially high-impact channels. The study will consider vulnerability discovery under a limited fault-injection budget.

4. Cross-Layer Fault Interaction Analysis

The research will investigate selected simultaneous faults involving channels from different network layers. The purpose is to examine whether combined cross-layer faults produce effects that differ from the corresponding individual fault impacts.

5. Vulnerability-Aware Robustness Enhancement

The identified fault-sensitive channels will be used to guide targeted fault-aware fine-tuning. The protection strategy will focus on selected vulnerable channels rather than uniformly applying additional fault-aware training to the complete network.

6. Budget-Aware Protection

The study will investigate different protection budgets to evaluate how the available protection resources affect robustness improvement, the number of protected channels, clean-model performance, and computational cost.

7. Comparative Evaluation

The proposed vulnerability-identification approach will be compared with selected baseline methods, including random fault selection, activation-based ranking, gradient/Taylor-based ranking, and DDPG-based search.

8. Experimental Scope

The experimental evaluation will focus on selected convolutional neural network architectures, primarily ResNet-18 and additionally VGG-16, using the CIFAR-10 dataset. The evaluation will consider controlled software fault injection and defined fault and protection budgets.

9. Evaluation Metrics

The study will evaluate vulnerability-discovery efficiency, faulted accuracy, clean accuracy, robustness improvement, and computational overhead. These metrics will be used to assess both the effectiveness and resource requirements of the proposed framework.

The scope does not include physical hardware fault-injection experiments, hardware redesign, runtime fault detection and recovery mechanisms, or deployment-specific hardware modifications. The proposed work is primarily concerned with software-based vulnerability analysis and targeted robustness enhancement of DNN models under simulated fault conditions.

3. LITERATURE SURVEY AND RESEARCH GAP

3.1 Introduction to DNN Reliability Research

Research on the reliability of Deep Neural Networks (DNNs) under hardware-induced faults has expanded alongside the deployment of neural-network models in safety-sensitive, embedded, and edge-computing environments. Recent research has examined how faults affecting weights, neurons, activation values, feature maps, and other internal computational elements can influence DNN prediction accuracy and overall reliability. The literature indicates that reliability assessment is increasingly being treated as a component-level problem rather than solely as an end-to-end model-performance problem.

A consistent observation across recent studies is that fault vulnerability is not uniformly distributed throughout a DNN. Empirical investigations involving convolutional networks, semantic-segmentation models, quantized DNN accelerators, and Vision Transformers have reported that a relatively small subset of internal components can account for a substantial proportion of the observed fault impact, while many other components exhibit comparatively limited sensitivity. This observation provides a basis for prioritizing reliability analysis and protection according to measured component criticality rather than treating all components equally. :contentReference[oaicite:1]{index=1}

Software-based fault injection has become an important methodology for evaluating DNN reliability because it allows controlled perturbations to be introduced into model computations without requiring dedicated physical fault-injection equipment. Recent surveys distinguish software-level simulation from RTL and hardware-based approaches in terms of fidelity and computational cost. Software injection provides a comparatively inexpensive mechanism for large-scale experimentation, whereas higher-fidelity hardware-level approaches can require substantially greater computational and experimental resources. This trade-off has motivated research into more selective and efficient fault-injection strategies. :contentReference[oaicite:2]{index=2}

Beyond conventional fault injection, recent work has investigated analytical vulnerability scoring, selective hardening, reinforcement-learning-based search, and fault-aware training. Vulnerability-ranking approaches have demonstrated that identifying high-impact channels or components can support selective protection with lower overhead than uniformly hardening an entire network. Reinforcement-learning approaches have further established that vulnerability assessment can be formulated as an adaptive search problem, in which the selection of subsequent fault-injection locations is influenced by information obtained from previous evaluations. :contentReference[oaicite:3]{index=3}

The literature also shows growing interest in converting vulnerability information into practical protection mechanisms. Existing approaches include selective duplication, pruning, checksum or consistency-based protection, hardware redundancy, selective lock-step execution, and reinforcement-learning-based protection masks. These studies demonstrate that reliability improvements do not necessarily require uniform protection of the complete model. Instead, protection can be concentrated on components or computational paths identified as more critical. However, the mechanisms and levels of protection vary considerably across studies, and many remain specific to particular architectures or hardware platforms. :contentReference[oaicite:4]{index=4}

An important limitation identified in the reviewed literature concerns the interaction between faults affecting different internal layers or channels. Most vulnerability-ranking methods evaluate components independently, while research on simultaneous fault injection indicates that multiple

faults can exhibit interaction or masking effects. The reviewed literature therefore provides evidence for individual component vulnerability but comparatively limited treatment of simultaneous, interacting faults across multiple internal layers of a single DNN. :contentReference[oaicite:5]{index=5}

Another recurring consideration is resource limitation. Several studies address selective protection, embedded deployment, FPGA-based acceleration, edge computing, or other constrained environments. However, the literature does not consistently combine efficient vulnerability discovery, cross-layer interaction analysis, and explicitly budget-constrained protection within a single end-to-end framework. The comparative assessment of the reviewed studies shows that cross-layer interaction modelling is particularly underrepresented: none of the fifteen shortlisted studies received strong coverage for this dimension, while only two provided partial coverage.

Based on this literature landscape, current DNN reliability research has established four important foundations relevant to the present study: fault vulnerability is non-uniform across internal components; software-based fault injection provides a practical mechanism for vulnerability assessment; adaptive or selective approaches can reduce the cost of reliability analysis and protection; and resource constraints are important for practical deployment. At the same time, the reviewed evidence indicates a need for further investigation into the relationship between efficient vulnerability discovery, cross-layer fault interactions, and budget-aware targeted protection.

These observations establish the research context for VulnShield-DNN and provide the basis for the detailed review of fault-injection techniques, vulnerability-assessment methods, reinforcement-learning approaches, fault-aware protection strategies, cross-layer analysis, and the resulting research gap presented in the following subsections.

3.2 Software-Based Fault Injection Techniques

Software-based fault injection is an established methodology for evaluating the reliability of Deep Neural Networks (DNNs) by introducing controlled perturbations into the model or its intermediate computations during execution. Unlike physical fault-injection experiments, software-based approaches do not require dedicated radiation sources, modified hardware platforms, or specialized accelerator instrumentation. They therefore provide a practical environment for studying DNN fault sensitivity across a large number of experiments.

Recent literature classifies DNN fault-injection approaches according to the location at which the fault is introduced, the representation affected, and the level of abstraction used for the experiment. Common injection granularities include weights, neurons, activation values, feature maps, and layer-level computational structures. The choice of injection granularity directly influences the type of vulnerability information that can be obtained. Component-level injection, for example, can provide fine-grained information about which internal elements are more sensitive to corruption, whereas higher-level injection may provide broader reliability characteristics with lower experimental complexity. :contentReference[oaicite:1]{index=1}

Software fault-injection frameworks such as PyTorchFI, PyTorchALFI, TensorFI+, and enpheePh have demonstrated the feasibility of introducing controlled perturbations into deep-learning models. These frameworks support different forms of runtime or execution-level fault injection and have been used to investigate the effects of faults on weights, neurons, activations, and intermediate computations. Their primary contribution is the ability to perform repeatable and

programmable reliability experiments without requiring permanent modification of the neural-network architecture. :contentReference[oaicite:2]{index=2}

The literature also distinguishes software-level injection from Register Transfer Level (RTL) simulation and physical hardware-based fault injection. Software-level approaches generally provide lower experimental cost and greater flexibility, making them suitable for large-scale exploratory studies. However, because they operate at a higher abstraction level, they may not reproduce every physical characteristic of an actual hardware fault. RTL and hardware-level approaches can provide greater fidelity to the underlying hardware implementation, but their computational and experimental cost can make exhaustive evaluation of large DNNs impractical. This establishes a trade-off between injection fidelity, experimental cost, and scalability. :contentReference[oaicite:3]{index=3}

The reviewed literature further demonstrates that fault vulnerability is highly non-uniform across a DNN. Empirical fault-injection studies on embedded semantic-segmentation networks have reported that vulnerability can be concentrated in a relatively small number of layers and feature maps, while many other channels exhibit comparatively limited impact on model performance. Similarly, studies involving quantized systolic-array-based DNN accelerators have shown that changes in numerical representation can modify the vulnerability landscape, causing the relative criticality of channels to vary with bit width. These observations support the use of selective rather than uniformly distributed fault analysis. :contentReference[oaicite:4]{index=4}

However, conventional fault-injection methodologies face a scalability problem. If every channel or potential fault location is evaluated independently, the number of required model executions can increase substantially with network depth and width. This is particularly relevant for modern DNNs containing large numbers of intermediate channels and computational operations. A systematic review of DNN hardware-reliability assessment methods identifies exhaustive fault injection as a limitation for large modern networks and highlights the need for more efficient methods of selecting injection locations. :contentReference[oaicite:5]{index=5}

Another limitation is that most conventional fault-injection studies evaluate components independently. Such evaluation provides useful information about individual fault sensitivity but does not necessarily describe the behavior of simultaneous faults affecting multiple components. Because DNN layers are computationally connected, faults introduced at different locations can potentially interact as their effects propagate through subsequent computations. The reviewed literature therefore identifies the absence of comprehensive cross-layer or cross-component interaction modelling as an important limitation of existing approaches. :contentReference[oaicite:6]{index=6}

For the present work, software-based fault injection provides the experimental foundation for channel-level vulnerability analysis. The proposed implementation focuses on controlled channel faults within convolutional DNNs. Specifically, the project synopsis defines a channel stuck-at-zero fault in which the selected channel output is temporarily replaced with zero using a PyTorch forward hook while preserving the original network architecture and tensor dimensions. This provides a controlled mechanism for measuring the effect of individual channel faults on model behaviour. :contentReference[oaicite:7]{index=7}

The limitation of exhaustive software fault injection motivates the subsequent investigation of intelligent fault-location selection. Rather than evaluating all possible channels with equal priority, the proposed research investigates whether reinforcement-learning-based search can direct a limited number of fault-injection experiments toward channels that exhibit greater measured fault impact. Thus, software fault injection is treated not as the complete vulnerability-detection solution, but as the controlled experimental mechanism through which a vulnerability-search strategy can obtain feedback.

Overall, the literature establishes software-based fault injection as a practical and flexible approach for DNN reliability assessment, while also identifying three important limitations: the abstraction gap between software simulation and physical hardware behaviour, the computational cost of exhaustive injection, and the predominantly independent treatment of fault locations. These limitations provide the technical motivation for the subsequent investigation of DNN vulnerability-assessment and ranking methods.

3.3 DNN Vulnerability Assessment Methods

The literature shows that DNN vulnerability assessment has progressed beyond exhaustive fault injection toward methods that explicitly score, rank, and prioritize internal components according to their measured or estimated sensitivity to faults. This development is motivated by the observation that vulnerability is generally non-uniform across a network, making selective assessment and protection potentially more efficient than treating all components equally.

A direct approach to vulnerability assessment is to perform a limited number of fault-injection trials for individual filters or channels and calculate a vulnerability score from the observed degradation. Ahmadilivani et al. (2024), for example, proposed per-filter or per-channel vulnerability scoring based on fault-injection experiments and subsequently used the ranking to selectively duplicate highly vulnerable output channels. Their work demonstrates an important transition from vulnerability measurement to actionable protection: vulnerability information can be used to prioritize only the components that contribute most strongly to observed degradation.

Analytical vulnerability assessment provides another approach for reducing dependence on exhaustive fault injection. Methods such as DeepVigor estimate vulnerability using analytical reliability factors and vulnerability ranges associated with DNN bits, neurons, and layers. Such approaches are useful because they can provide vulnerability information without requiring every possible fault location to be physically or computationally injected. However, analytical estimates may depend on assumptions about the network structure, fault model, and relationship between internal values and final prediction behaviour.

The literature also demonstrates that vulnerability ranking is not restricted to conventional convolutional networks. Ahsaei and Raji's work on Vision Transformers performs fine-grained sensitivity analysis across encoder layers, weight-matrix types, and attention-head dimensions. Their findings indicate that vulnerability can also be concentrated in specific internal components of transformer architectures rather than being uniformly distributed throughout the model. However, the ranking procedure remains architecture-specific, leaving open the question of whether a common vulnerability-discovery mechanism can be applied across different architecture families.

Another important observation is that vulnerability ranking can be used to reduce protection overhead. The selective-hardening study reviewed in the literature demonstrates that protecting highly ranked channels can recover a large proportion of the accuracy loss associated with faults

while requiring substantially less overhead than protecting the complete network. Similar criticality-aware approaches have subsequently appeared in hardware and edge-oriented DNN reliability research. These studies establish that vulnerability ranking has practical value when it is connected to a protection mechanism.

Despite these advances, existing vulnerability-ranking approaches have several limitations relevant to the present research. First, many methods evaluate channels or components independently, producing individual criticality scores without explicitly modelling how two vulnerable components may interact when faults occur simultaneously. The reviewed literature identifies the absence of scalable interaction-aware vulnerability discovery as a recurring limitation.

Second, vulnerability-ranking methods differ considerably in the information used to construct their rankings. Fault-injection-based scores directly measure the effect of corruption, whereas activation-, gradient-, or analytically derived measures provide indirect estimates of component importance. Consequently, a component that appears important according to a normal-operation statistic is not necessarily the component whose corruption produces the greatest fault-induced degradation. This distinction motivates direct comparison between heuristic rankings and rankings obtained from controlled fault-injection feedback.

Third, ranking quality should not be evaluated solely by how well a method orders components according to a particular scoring criterion. A practically useful ranking should also demonstrate that selecting the highest-ranked components leads to measurable improvement when those components are subsequently protected. The reviewed literature provides evidence for this analysis-to-protection relationship, but the approaches differ in their protection mechanisms and generally do not evaluate the complete process under a common explicit protection budget.

The present project therefore treats vulnerability assessment as a channel-selection and prioritization problem rather than simply as a fault-detection problem. The proposed evaluation will compare random search, activation Top-K ranking, gradient/Taylor-based ranking, layer-wise DDPG, and TD3-based search within a common experimental setting. The objective is to determine whether the TD3 strategy can identify high-impact channels efficiently under a limited fault-injection budget and whether its resulting ranking provides useful information for subsequent protection.

Thus, the reviewed literature establishes that component-level vulnerability ranking is feasible and useful, but also indicates that existing approaches do not simultaneously provide efficient adaptive channel discovery, explicit cross-layer interaction analysis, and budget-constrained vulnerability-guided protection. These limitations provide the transition to the next subsection, which examines reinforcement-learning approaches for adaptive vulnerability discovery.

3.4 Reinforcement Learning for DNN Vulnerability Detection

Reinforcement learning (RL) provides an adaptive approach to vulnerability discovery in Deep Neural Networks by treating the selection of fault locations as a sequential decision-making problem. Instead of evaluating fault locations according to a fixed ordering or predetermined heuristic, an RL agent can use information obtained from previous fault-injection evaluations to influence subsequent selections. This makes RL relevant to DNN vulnerability assessment where the number of possible fault locations is large and the available fault-injection budget is limited.

Recent literature provides direct evidence that reinforcement learning can be applied to DNN reliability assessment. A 2025 study published in *Sensors* proposes a three-stage approach consisting of layer-wise resiliency profiling through fault-injection simulation, reinforcement-learning-based learning of bit-protection masks, and transfer learning for reuse across different

DNN architectures. The reported results show that the learned protection masks can outperform selected fixed protection schemes under an equal protection budget. The study also reports non-monotonic vulnerability across network depth, with some intermediate ResNet-18 layers exhibiting greater sensitivity than earlier or later layers.

The reviewed project literature also identifies a 2026 IEEE Transactions on Reliability study as the base work for the present project. That work develops a deep reinforcement-learning framework for identifying vulnerable neurons through guided fault injection. Its significance is that vulnerability discovery is treated as an adaptive search process rather than as a purely exhaustive evaluation. This establishes the feasibility of using reinforcement learning to direct fault-injection experiments toward vulnerable internal components.

However, the existing reinforcement-learning approaches differ from the proposed formulation in their level of analysis. The Sensors study operates primarily at bit and layer granularity and uses learned protection masks, while the selected base work focuses on vulnerable neurons. The present project instead investigates vulnerability at the convolutional channel level and proposes TD3 as the specific reinforcement-learning algorithm for selecting fault-injection locations. The project will also compare TD3 with a layer-wise DDPG baseline to determine whether the stability characteristics of TD3 provide measurable benefits for vulnerability discovery.

The choice of TD3 is therefore not based on the claim that reinforcement learning itself is novel for DNN reliability. RL-based vulnerability search has already been demonstrated in prior research. The research question is instead whether a TD3-based continuous-control strategy can provide an effective mechanism for channel-level fault-location selection under a constrained injection budget and how its performance compares with DDPG and non-RL ranking methods within the same experimental setting.

An important consideration in RL-based vulnerability discovery is the computational cost of training the agent. Although adaptive search can reduce the number of unnecessary fault-injection evaluations, the RL process itself requires repeated environment interactions and model updates. The reviewed literature specifically notes that reinforcement-learning training introduces non-trivial computational cost, which can partially offset the savings obtained by avoiding exhaustive fault injection. Consequently, RL-based vulnerability detection should be evaluated not only according to the quality of the channels discovered but also according to the number of fault-injection trials and computational resources required to obtain the ranking.

For the proposed framework, vulnerability discovery will therefore be formulated around controlled channel-level fault injection. The TD3 agent will select candidate channels for fault evaluation, receive feedback based on the observed effect of the injected fault, and progressively learn which channel locations are more likely to produce significant degradation. The resulting observations will be used to construct a ranked vulnerability map. The project synopsis specifies random search, activation Top-K ranking, gradient/Taylor ranking, and layer-wise DDPG as comparison methods, enabling TD3 to be evaluated against both heuristic and reinforcement-learning alternatives.

The literature also indicates that RL-based vulnerability discovery and protection are related but distinct tasks. An RL agent may be trained to identify vulnerable components, while a separate mechanism may use the resulting information to determine how those components should be protected. The present project explicitly connects these stages by using the TD3-derived vulnerability ranking as the basis for targeted fault-aware fine-tuning. This allows the usefulness of the ranking to be assessed through its downstream protection performance rather than treating vulnerability discovery as an isolated analytical task.

Thus, existing research establishes the feasibility of reinforcement-learning-based adaptive vulnerability search, but the reviewed literature leaves several aspects open for investigation: channel-level rather than neuron-, bit-, or layer-level discovery; direct comparison of TD3 with DDPG and heuristic rankings under a common fault-injection budget; explicit analysis of selected cross-layer interactions following vulnerability discovery; and validation of the resulting ranking through targeted protection under defined protection budgets. These aspects form the basis for the proposed reinforcement-learning component of VulnShield-DNN.

3.5 Fault-Aware Training and DNN Protection

Vulnerability assessment provides information about which components of a Deep Neural Network are sensitive to faults, but vulnerability identification alone does not improve the reliability of the model. A second research direction therefore focuses on converting vulnerability information into protection mechanisms that enable the DNN to tolerate or mitigate the effects of internal faults. The reviewed literature contains several approaches based on fault-aware training, selective redundancy, fault detection and masking, and hardware-level criticality-aware protection.

Fault-aware training incorporates simulated fault conditions into the training process so that the network can learn to maintain its prediction capability when internal computations are corrupted. Fernandes dos Santos et al. investigate transient-fault-aware design and training for improving DNN reliability, while Cavagnero et al. study fault-aware training as a method for enhancing reliability without introducing additional inference-time hardware overhead. These approaches establish that exposure to representative fault conditions during training can be used as a mechanism for improving DNN tolerance to transient computational errors.

Another approach is selective protection based on measured component vulnerability. Ahmadilivani et al. use per-filter or per-channel vulnerability scores obtained from fault-injection experiments and apply selective duplication to highly vulnerable output channels. Their results indicate that protecting only the most vulnerable channels can recover a substantial portion of the accuracy degradation while requiring less overhead than protecting the complete network. This provides evidence that vulnerability information can be translated into targeted protection rather than uniform hardening.

Protection mechanisms have also been investigated at the hardware-architecture level. Burel et al. propose checksum- and consistency-based detection and masking at selected points within a semantic-segmentation network. Syed et al. investigate a reconfigurable CNN accelerator in which redundancy is selectively enabled for reliability-critical branches. Similarly, Sun et al.'s Aphelios employs selective lock-step execution for computation paths identified as highly critical. These studies demonstrate that reliability protection can be concentrated on selected computational paths to reduce the overhead associated with full redundancy.

Reinforcement learning has also been used to connect vulnerability assessment with protection. The reviewed Sensors study uses an RL agent to learn per-layer bit-protection masks and reports improved protection compared with selected fixed protection schemes under an equal protection budget. This demonstrates that learned criticality information can be used not only for vulnerability assessment but also for determining which components should receive protection. However, the approach operates primarily at bit and layer granularity and does not explicitly model simultaneous interactions between vulnerable channels across different internal layers.

The literature therefore demonstrates several distinct protection strategies:

1. Fault-aware training, where the model is exposed to simulated faults during training.
2. Selective duplication or redundancy, where additional computation is applied to highly vulnerable components.
3. Fault detection and masking, where selected computational locations are monitored and corrected or masked.
4. Hardware-level selective protection, where redundancy or lock-step execution is activated only for reliability-critical paths.
5. Reinforcement-learning-based protection, where learned criticality information determines the components or bits receiving additional protection.

Although these approaches differ in implementation, they share a common principle: **protection should be concentrated on components that have demonstrated greater reliability impact** rather than being uniformly applied across the complete network. This principle is directly relevant to the proposed vulnerability-guided protection strategy.

A limitation across the reviewed approaches is that vulnerability discovery and protection are not consistently formulated as a single resource-constrained optimization problem. Several studies demonstrate selective protection, but the selection mechanism, protection technology, and resource constraints vary according to the target architecture. The literature review identifies the absence of an end-to-end framework that explicitly connects efficient vulnerability discovery, cross-layer interaction analysis, and budget-constrained protection.

The distinction between protection during training and protection during inference is also important. Fault-aware training attempts to modify the learned model so that it becomes more tolerant of specified fault conditions, whereas hardware redundancy, masking, and lock-step approaches introduce additional mechanisms during execution. The proposed work focuses on the former: vulnerability information obtained through software fault injection is used to select channels for targeted fault-aware fine-tuning. The objective is to investigate whether selective training can improve robustness while avoiding the computational cost of applying additional fault-aware training uniformly to all channels.

In the proposed framework, the protection process is explicitly connected to the vulnerability ranking generated by the TD3-based search. Selected vulnerable channels are exposed to the defined channel fault during fine-tuning, while clean and faulted training objectives are combined so that the model is encouraged to preserve normal predictive performance while improving tolerance to the simulated fault. The project further specifies equal protection budgets of 1%, 3%, 5%, and 10% for comparing random, heuristic, DDPG-guided, and TD3-guided channel selection.

The usefulness of a vulnerability ranking is therefore evaluated not only by its ability to identify channels with high measured fault impact but also by whether protecting those channels produces measurable robustness improvement under held-out fault conditions. This creates a direct link between vulnerability analysis and protection performance.

Overall, existing research establishes the effectiveness of fault-aware training and selective protection as approaches for improving DNN reliability. However, the reviewed literature does not provide a unified treatment of efficient channel-level vulnerability discovery, explicit cross-layer fault interaction analysis, and budget-aware vulnerability-guided fine-tuning. This limitation

provides the basis for examining simultaneous and cross-layer fault behaviour in the following subsection.

3.6 Simultaneous and Cross-Layer Fault Analysis

Fault-injection studies have traditionally evaluated DNN components independently, measuring the effect of a fault introduced into one weight, neuron, channel, or layer at a time. This approach provides useful information about individual component sensitivity, but it does not necessarily describe the behaviour of multiple faults occurring simultaneously. Since DNN computations are sequentially connected across layers, faults introduced at different internal locations can interact as their effects propagate through subsequent computations.

Recent research has explicitly investigated simultaneous fault injection as a means of improving the scalability and realism of DNN reliability assessment. Tonetto et al. study simultaneous fault injection and report that multiple faults can interact and produce effects such as masking, meaning that the impact observed when faults are evaluated together may differ from the impact inferred from independent fault evaluations. This observation is important because independent vulnerability scores may not fully represent the behaviour of a system exposed to multiple simultaneous faults.

The distinction between individual and simultaneous fault effects can be expressed using the individual degradation of two selected channels and their combined degradation. Let $D(i)$ denote the measured degradation caused by faulting channel i individually, and $D(i,j)$ denote the degradation observed when channels i and j are faulted simultaneously. If the combined degradation is substantially different from the value expected from the individual effects, this provides evidence of an interaction between the two fault locations. The exact interaction metric and interpretation must be established experimentally according to the selected fault model and evaluation metric.

Cross-layer analysis is particularly relevant because channels belonging to different layers participate in different stages of the network's feature transformation. A fault in an earlier layer can alter the representation supplied to later layers, while a fault in a later layer affects an already transformed representation. When faults occur simultaneously at these different locations, their effects may propagate and combine in ways that cannot be inferred solely from independent channel vulnerability scores.

The reviewed literature identifies this issue as an important unresolved area. The systematic reliability review included in the literature survey reports that most existing approaches evaluate internal components independently and do not provide scalable interaction-aware vulnerability discovery. Similarly, the vulnerability-scoring methods reviewed in Section 3.3 primarily rank individual channels without explicitly modelling how simultaneous faults in different layers may interact.

Some existing work provides partial evidence related to multiple computational paths or distributed systems. For example, research on distributed edge inference considers interactions between faulty computation paths across multiple devices. However, such interaction is different from the intra-network cross-layer interaction considered in VulnShield-DNN. The proposed analysis focuses specifically on simultaneous faults affecting selected channels belonging to different layers or network blocks within the same DNN.

The literature review of fifteen shortlisted studies provides a particularly clear observation: none of the reviewed studies explicitly models simultaneous, interacting faults across multiple internal

layers of a single DNN. This makes cross-layer interaction one of the most consistently under-addressed dimensions in the reviewed research.

This gap is directly relevant to the proposed framework. After the vulnerability-discovery stage, VulnShield-DNN will select high-ranked fault-sensitive channels from different layers and evaluate selected combinations under simultaneous fault conditions. The purpose is not to assume that cross-layer interaction exists or that it necessarily increases degradation, but to experimentally determine whether the combined effect differs from the effects measured for the corresponding individual faults.

The proposed analysis will therefore investigate questions such as whether two individually vulnerable channels remain independently influential when faulted together, whether one fault masks the effect of another, and whether certain combinations produce greater degradation than would be expected from their individual measurements. These observations will provide evidence about the relationship between individual channel criticality and multi-location fault behaviour.

The analysis is intentionally restricted to selected combinations rather than exhaustive evaluation of every possible pair of channels. Exhaustive pairwise or higher-order fault injection would introduce a combinatorial increase in the number of experiments, conflicting with the resource-constrained objective of the project. The vulnerability ranking generated during the preceding discovery stage is therefore used to restrict cross-layer analysis to selected high-impact channels.

This design creates a sequential relationship between the research components:

Fault Injection

- Individual Channel Vulnerability Measurement
- Vulnerability Ranking
- Selection of High-Risk Channels from Different Layers
- Simultaneous Cross-Layer Fault Injection
- Measurement of Combined Fault Impact.

This relationship is important because the cross-layer analysis is not intended to replace vulnerability detection. Instead, it builds upon the vulnerability ranking to investigate whether individually identified high-risk components exhibit additional interaction behaviour when faults occur simultaneously.

The reviewed literature therefore establishes the importance of simultaneous fault effects and identifies a substantial gap in explicit intra-network cross-layer interaction modelling. VulnShield-DNN addresses this gap by experimentally analysing selected simultaneous faults among ranked vulnerable channels from different network layers and comparing their combined effects with the corresponding individual fault effects. This analysis forms one of the principal research components of the proposed framework.

3.7 Adversarial Sensitivity and Fault Vulnerability

Hardware-fault vulnerability and adversarial sensitivity represent two different mechanisms through which a Deep Neural Network can experience degraded prediction performance. Hardware-fault vulnerability concerns errors introduced into the computational process, such as corruption of weights or intermediate activations, whereas adversarial sensitivity concerns intentionally crafted input perturbations designed to alter the model's prediction. Although the underlying causes differ, both forms of sensitivity can reveal internal components whose behaviour has a strong influence on the final model output.

Recent literature has examined the relationship between deliberate fault-injection attacks and hardware-reliability analysis. A 2025 systematic review of disruptive attacks against artificial neural networks considers hardware-fault-injection attacks alongside detection and protection mechanisms. The review reports that fault-injection attacks remain a relevant security concern, particularly in shared and multi-tenant FPGA environments, and observes that many reported protection mechanisms are reactive and detection-oriented rather than proactively identifying and hardening highly exploitable components.

This literature provides an important conceptual connection between reliability and security research. A component that is highly sensitive to an accidental hardware fault may also represent a potentially important target for a deliberate fault-injection attack. Conversely, sensitivity to adversarial input perturbations does not necessarily imply sensitivity to internal hardware faults. The two forms of vulnerability therefore should not be treated as equivalent.

The present project investigates this relationship through a supporting comparison between hardware-fault-sensitive channels and adversarially sensitive channels. The project synopsis specifies the use of Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD) adversarial examples for this comparison. The analysis will use the same convolutional channel representation employed for the fault-vulnerability study, allowing channel-level sensitivity to be compared using a common internal representation.

Under this comparison, channels can be categorized according to their observed sensitivity patterns. A channel may be identified as fault-only vulnerable if it exhibits significant degradation under simulated hardware faults but comparatively limited sensitivity under the evaluated adversarial inputs. It may be adversarial-only sensitive if the opposite pattern is observed. A channel exhibiting sensitivity under both conditions can be classified as dual vulnerable, while a channel showing limited response to both evaluated perturbation types can be considered relatively robust within the experimental conditions.

This comparison is intended to determine whether fault-sensitive channels and adversarially sensitive channels overlap systematically or represent substantially different internal vulnerability structures. The result should not be interpreted as evidence that adversarial sensitivity is a substitute for hardware-fault testing. Instead, the experiment provides an additional characterization of whether the channels identified through hardware-fault injection also exhibit sensitivity to input-level adversarial perturbations.

The distinction is important because the proposed VulnShield-DNN framework is primarily concerned with **hardware-fault vulnerability**, not adversarial-attack mitigation. The adversarial experiment is therefore a supporting analysis rather than a primary protection mechanism. Its purpose is to provide additional evidence about the characteristics of the channels identified by the proposed vulnerability-ranking procedure.

The literature also indicates that the security and reliability communities have historically used different terminology and evaluation methodologies even when studying related internal weaknesses. Hardware-fault research generally focuses on dependability, fault tolerance, and reliability, while adversarial-machine-learning research focuses on attack success, robustness, and security. The reviewed systematic literature suggests that these research areas can overlap in the internal vulnerabilities they expose, creating an opportunity for comparative analysis without conflating their underlying mechanisms.

For VulnShield-DNN, the comparison will therefore remain experimentally bounded. FGSM and PGD will be used to generate adversarial examples, while the hardware-fault analysis will use the

defined software fault model and channel-level fault injection procedure. The resulting channel-level sensitivity measurements will be compared rather than assuming that one type of perturbation predicts the other.

The significance of this analysis is twofold. First, it can determine whether the vulnerability ranking obtained from hardware-fault experiments identifies channels that are also sensitive to adversarial perturbations. Second, it can reveal channels whose sensitivity is specific to one perturbation mechanism. Such observations can help distinguish general internal criticality from fault-model-specific vulnerability.

Overall, existing research establishes that hardware fault injection has both reliability and security relevance, while adversarial sensitivity provides a complementary perspective on DNN robustness. However, the reviewed literature does not establish a general equivalence between hardware-fault-sensitive channels and adversarially sensitive channels. VulnShield-DNN therefore treats their comparison as a supporting experiment using a common channel representation, while keeping hardware-fault vulnerability identification and fault-aware protection as the primary focus of the proposed research.

3.8 Comparative Analysis of Existing Approaches

The comparative analysis below is based on the 15 shortlisted studies documented in the literature-survey file. The comparison focuses specifically on methodology, principal findings, and the limitations relevant to VulnShield-DNN. The source also evaluates the studies against four project pillars: efficient vulnerability discovery, cross-layer interaction modelling, budget-aware protection, and edge/resource-constrained deployment.

No.	Study / Year	Venue	Methodology	Principal Finding	Gap Relative to VulnShield-DNN
1	Ahmadilivani et al. (2024)	ACM Computing Surveys	Systematic review of 100+ DNN hardware-reliability studies	DNN vulnerability is non-uniform across layers and channels; interaction-aware assessment remains limited	No explicit cross-layer interaction model
2	Yu, Tan & Huang (2024)	ACM Computing Surveys	Classification of fault-injection tools according to fidelity, cost, and scope	Software injection is inexpensive but may underestimate hardware effects; RTL/HW approaches are more accurate but costly	No efficient middle-ground sampling strategy
3	Gutiérrez-Zaballa et al. (2024)	Journal of Systems Architecture	Layer-wise and bit-position fault injection in embedded segmentation networks	Vulnerability is concentrated in particular layers and feature maps; many channels	Architecture-specific analysis with no integrated protection mechanism

No.	Study / Year	Venue	Methodology	Principal Finding	Gap Relative to VulnShield-DNN
				show limited sensitivity	
4	Taheri et al. (2024)	DATE 2024	Activation-fault injection in quantized systolic-array DNN accelerators	Quantization changes the vulnerability distribution and shifts critical channels with bit-width	No cross-layer interaction or integrated budget-aware protection
5	Mesa Mesa & Pérez Holguín (2025)	EIEI-ACOFI 2025	Fault injection on hardware-implemented DNNs	A small subset of layers and parameters can dominate performance degradation	Single-architecture analysis without interaction modelling
6	Ahmadilivani et al. (2024)	IEEE IOLTS 2024	Per-channel vulnerability scoring followed by selective duplication and pruning	Highly vulnerable channels can be selectively protected with lower overhead than full-network protection	No explicit constrained optimization for protection and no cross-layer interaction modelling
7	Ahsaei & Raji (2025) — LOST-ViT	Journal of Systems Architecture	Fine-grained sensitivity analysis of Vision Transformer components	Vulnerability is concentrated in particular attention-head dimensions and weight types	Ranking remains architecture-specific
8	RL Bit-Masking Study (2025)	Sensors	RL-based layer-wise resiliency profiling and bit-protection masks with transfer learning	Learned protection masks outperform selected fixed protection schemes under equal budgets	Bit/layer granularity; cross-layer channel interaction is not modelled; RL training has computational cost
9	Burel et al. (2024)	Microelectronics Reliability	Checksum- and consistency-based detection and masking	Strategically placed detection and masking can capture important faults without complete instrumentation	No explicit budget-constrained optimization
10	Syed et al. (2024)	IEEE Access	Reconfigurable FPGA CNN accelerator with	Reliability close to full redundancy can be achieved	Hardware- and architecture-specific; no

No.	Study / Year	Venue	Methodology	Principal Finding	Gap Relative to VulnShield-DNN
			selective redundancy	with lower overhead by protecting critical branches	cross-layer channel model
11	Sun et al. (2025) — Aphelios	ACM GLSVLSI	Selective lock-step execution using criticality estimation	Critical computation paths can be selectively duplicated to reduce redundant computation and energy	Hardware-specific and lacks cross-layer vulnerability modelling
12	Distributed Edge Inference Study (2025)	Scientific Reports	Criticality-aware protection for distributed CNN inference across edge/IoT devices	Selective runtime compensation can reduce the need for uniform redundancy	Interaction concerns are inter-device rather than intra-network cross-layer
13	Adversarial/Fault Review (2025)	Elsevier	Review of disruptive attacks and fault-injection security mechanisms	Hardware fault injection has both reliability and security implications	Does not provide the proposed channel-level vulnerability-to-protection pipeline
14	DCIS-Class Architecture Study	—	Reliability-oriented architecture for resource-constrained DNN deployment	Demonstrates the importance of reliability under constrained hardware resources	Does not integrate adaptive vulnerability discovery with cross-layer analysis
15	Activation-Function Reliability Study	—	Evaluation of activation-function behaviour under fault conditions	Activation choice can influence DNN reliability	Does not provide channel-level adaptive vulnerability discovery or integrated protection

Table 3.1: Summary of Reviewed Literature (2024–2026)

The source literature explicitly uses this comparison to distinguish what has already been demonstrated from what remains unresolved. In particular, several studies independently establish non-uniform vulnerability, while selective protection and reinforcement-learning-based approaches establish that vulnerability information can be converted into practical protection mechanisms.

The reviewed studies were additionally assessed against four dimensions:

- P1 — Efficient vulnerability discovery
- P2 — Cross-layer/cross-channel interaction modelling
- P3 — Budget-aware selective protection
- P4 — Edge/resource-constrained deployment

No.	Study	P1: Efficient Discovery	P2: Cross-Layer Interaction	P3: Budget-Aware Protection	P4: Edge Deployability
1	Ahmadilivani et al. — Survey	Partial	No	No	Partial
2	Yu, Tan & Huang — Survey	Partial	No	No	No
3	Gutiérrez-Zaballa et al.	Partial	No	No	Strong
4	Taheri et al.	Partial	No	No	Strong
5	Mesa Mesa & Pérez Holguín	Partial	No	No	Partial
6	Ahmadilivani et al. — IOLTS	Strong	Partial	Strong	Partial
7	LOST-ViT	Strong	No	No	Partial
8	RL Bit-Masking	Strong	No	Strong	Strong
9	Burel et al.	Partial	No	Strong	Strong
10	Syed et al. — FPGA Accelerator	No	No	Strong	Strong
11	Sun et al. — Aphelios	Partial	No	Strong	Strong
12	Distributed Edge Inference	No	Partial	Strong	Strong
13	Adversarial/Fault Review	No	No	No	Partial
14	DCIS-Class Architecture	No	No	Strong	Strong
15	Activation-Function Reliability	No	No	Partial	Strong

Table 3.2: Comparison Against the Four Project Pillars

The aggregated analysis provides an important result: efficient vulnerability discovery and budget-aware protection are already addressed to varying degrees in the literature, whereas explicit cross-layer interaction modelling is substantially less developed. The source reports that none of the fifteen studies provides strong coverage of the cross-layer interaction pillar, with only two providing partial coverage.

Therefore, the comparison does not support the claim that VulnShield-DNN is the first project to use reinforcement learning, vulnerability ranking, or selective protection individually. Those components already have precedents.

Instead, the comparative evidence points toward the combination and integration of these mechanisms:

Efficient channel-level vulnerability discovery → explicit cross-layer fault-interaction analysis → vulnerability-guided protection under defined budgets → joint evaluation of robustness and computational efficiency.

This distinction is important because it gives the project a defensible research position rather than relying on an overly broad novelty claim. The literature review identifies the lack of an end-to-end framework combining these dimensions as the central unresolved area.

3.9 Research Gap Analysis

The literature reviewed in the preceding sections establishes that Deep Neural Network reliability under hardware-induced faults has been investigated from several complementary perspectives, including software-based fault injection, vulnerability ranking, reinforcement-learning-based search, selective protection, fault-aware training, and resource-aware deployment. However, these research directions are generally addressed independently or with limited integration. The comparison of the fifteen shortlisted studies provides the basis for identifying the specific research gaps addressed by VulnShield-DNN.

1. Gap in Efficient Channel-Level Vulnerability Discovery

Existing studies demonstrate that DNN vulnerability is non-uniform and that selected internal components can have substantially greater influence on fault-induced degradation than others. Several approaches use fault injection, analytical scoring, activation statistics, or gradient-based measures to rank vulnerable components. Reinforcement-learning approaches have also demonstrated adaptive vulnerability discovery.

However, the reviewed literature does not establish a common channel-level framework that systematically compares TD3-based search with random, activation-based, gradient/Taylor-based, and DDPG-based selection under an identical fault-injection budget. This creates a need to evaluate whether adaptive TD3-based exploration can improve vulnerability-discovery efficiency at the channel level.

2. Gap in Explicit Cross-Layer Fault-Interaction Modelling

Cross-layer interaction represents the strongest gap identified in the literature review. Existing vulnerability-ranking approaches predominantly evaluate individual components, while studies on simultaneous fault injection demonstrate that multiple faults can exhibit interaction or masking effects.

The literature comparison indicates that none of the fifteen shortlisted studies provides strong coverage of explicit simultaneous fault interaction across multiple internal layers of a single DNN. Only limited or partial treatment of interaction is observed in a small number of studies, and these interactions are not equivalent to the proposed intra-network cross-layer channel analysis.

Therefore, there is a need to experimentally investigate whether simultaneously faulting selected vulnerable channels from different layers produces degradation that differs from the effects measured when those channels are faulted individually.

3. Gap in Integration Between Vulnerability Discovery and Protection

Existing research demonstrates both vulnerability identification and selective protection. Vulnerability scores have been used to guide duplication, redundancy, masking, lock-step execution, and other protection mechanisms. Reinforcement learning has also been used to generate protection masks.

However, these approaches do not consistently establish a unified pipeline in which vulnerability discovery, cross-layer interaction analysis, and targeted protection are evaluated as connected stages. In particular, the relationship between a channel's measured vulnerability and its usefulness as a target for fault-aware fine-tuning requires further experimental investigation.

4. Gap in Channel-Level Budget-Aware Robustness Enhancement

Several studies investigate selective protection under resource constraints, including equal protection budgets and selective redundancy. However, the reviewed approaches employ different protection mechanisms and levels of abstraction, including bits, layers, branches, hardware paths, and channels.

A specific gap remains in evaluating whether a vulnerability ranking obtained through adaptive channel-level fault injection can be used to allocate a predefined fine-tuning budget toward the most fault-sensitive channels while maintaining clean-model performance.

5. Gap in Unified Evaluation of Discovery Efficiency and Protection Effectiveness

Existing studies commonly evaluate either vulnerability identification or protection effectiveness, while fewer studies evaluate the complete relationship between the cost of discovering vulnerabilities and the robustness obtained after protecting the discovered components.

A vulnerability-discovery method that identifies highly vulnerable channels but requires excessive fault-injection evaluations may not provide a practical advantage. Similarly, a protection method that improves faulted accuracy at the expense of substantial clean accuracy degradation may not provide a useful reliability solution.

Therefore, there is a need for joint evaluation using measures such as vulnerability-discovery efficiency, faulted accuracy, clean accuracy, robustness improvement, and computational overhead.

6. Gap in Cross-Model and Supporting Sensitivity Analysis

The reviewed literature includes studies across CNNs, segmentation networks, quantized DNN accelerators, and Vision Transformers. However, vulnerability characteristics can depend on architecture, numerical representation, and deployment conditions.

The proposed work therefore includes evaluation primarily on ResNet-18 and additionally on VGG-16 to determine whether the observed vulnerability-discovery and protection behaviour is limited to a single CNN architecture. A supporting comparison with FGSM and PGD adversarial sensitivity is also included to examine whether channels identified as hardware-fault-sensitive exhibit similar or different sensitivity to input-level adversarial perturbations.

7. Consolidated Research Gap

Based on the reviewed evidence, the principal research gap is not the absence of reinforcement learning, vulnerability ranking, selective protection, or fault-aware training as individual techniques. Each of these directions has existing research support.

The principal gap lies in their systematic integration with explicit cross-layer fault-interaction analysis and defined resource constraints. The reviewed literature does not provide a sufficiently unified framework that:

- efficiently identifies fault-sensitive channels using adaptive search;
- experimentally analyses simultaneous faults across different internal layers;
- uses the resulting vulnerability information to prioritize targeted fault-aware protection;
- evaluates protection under explicitly defined budgets; and
- jointly measures vulnerability-discovery efficiency, robustness improvement, clean-model performance, and computational overhead.

VulnShield-DNN is designed to investigate this combined research problem. The framework therefore positions its contribution around the integration and experimental evaluation of these components rather than claiming that any individual component, such as reinforcement learning or fault-aware training, is itself new.

3.10 Novelty and Research Contributions

The literature review indicates that the individual techniques used in VulnShield-DNN are not independently new. Software-based fault injection, vulnerability ranking, reinforcement-learning-based vulnerability search, selective protection, fault-aware training, and resource-aware reliability analysis have all been investigated in previous research. Therefore, the novelty of the proposed work is defined by the integration, experimental formulation, and evaluation of these components rather than by claiming any individual technique as a new invention.

The proposed research contributes the following aspects:

1. TD3-Based Channel-Level Vulnerability Discovery

The proposed framework investigates Twin Delayed Deep Deterministic Policy Gradient (TD3) for adaptive selection of fault-injection locations at the internal channel level of convolutional DNNs. The objective is to determine whether TD3 can efficiently discover high-impact channels under a constrained fault-injection budget.

The novelty claim is therefore not the use of reinforcement learning itself, but the investigation of TD3 as a channel-level vulnerability-search mechanism within the proposed experimental framework and its direct comparison with random, activation-based, gradient/Taylor-based, and DDPG-based approaches.

2. Explicit Cross-Layer Fault-Interaction Analysis

A principal contribution of the proposed work is the explicit investigation of simultaneous faults affecting selected vulnerable channels belonging to different network layers.

Existing vulnerability-ranking approaches predominantly assess individual components, while the literature review identifies limited treatment of simultaneous interacting faults across multiple internal layers of a single DNN. VulnShield-DNN therefore uses the vulnerability ranking obtained during the discovery stage to select cross-layer channel combinations and experimentally measure their combined fault impact.

The framework does not assume that cross-layer faults necessarily amplify one another. Instead, it experimentally investigates whether combined faults produce additive, masking, amplified, or otherwise different behaviour relative to their individual fault effects.

3. Vulnerability-Guided Protection Pipeline

The proposed framework connects vulnerability discovery with robustness enhancement through an explicit sequential pipeline:

Software Fault Injection

- TD3-Based Vulnerability Discovery
- Vulnerability Ranking
- Cross-Layer Interaction Analysis
- Vulnerability-Guided Fault-Aware Fine-Tuning
- Protected DNN

This integration provides a common experimental framework in which the usefulness of vulnerability identification can be evaluated through its downstream protection performance.

4. Budget-Aware Channel Selection for Protection

The proposed work investigates targeted protection under predefined protection budgets rather than assuming that the entire network can be protected uniformly.

Different protection budgets will be used to determine how many channels should receive fault-aware fine-tuning and how the choice of protected channels influences faulted accuracy, clean accuracy, robustness improvement, and computational cost.

The contribution is therefore the experimental study of the relationship between vulnerability ranking quality and protection effectiveness under controlled channel-selection budgets.

5. Joint Evaluation of Vulnerability Discovery and Robustness Enhancement

The proposed framework evaluates the vulnerability-discovery stage and the protection stage using connected performance measures.

The evaluation will consider:

- Vulnerability-discovery efficiency
- Number of fault-injection evaluations
- Faulted accuracy
- Clean-data accuracy
- Robustness improvement
- Protection coverage
- Computational overhead

This allows the study to determine not only whether vulnerable channels can be identified, but also whether efficiently identified channels provide useful targets for subsequent protection.

6. Comparative Evaluation Under a Common Experimental Setting

The proposed work establishes a common evaluation environment for comparing multiple vulnerability-selection strategies:

- Random fault selection
- Activation-based Top-K ranking
- Gradient/Taylor-based ranking
- Layer-wise DDPG
- TD3-based selection

The comparison is intended to determine differences in vulnerability discovery and downstream protection effectiveness while controlling the fault model, dataset, network architecture, injection budget, and evaluation procedure.

7. Supporting Comparison with Adversarial Sensitivity

As an additional analysis, the project compares channels identified through hardware-fault vulnerability assessment with channels exhibiting sensitivity to FGSM and PGD adversarial perturbations.

This comparison is not presented as the primary contribution or as evidence that hardware faults and adversarial attacks are equivalent. Instead, it is used to investigate whether the two perturbation mechanisms identify overlapping or distinct internal sensitivity patterns.

8. Cross-Architecture Evaluation

The proposed experiments will primarily use ResNet-18 and additionally consider VGG-16 on CIFAR-10. This provides an opportunity to examine whether the observed vulnerability-discovery and protection behaviour is specific to one architecture or can be observed across more than one convolutional DNN architecture.

9. Consolidated Research Contribution

The central contribution of VulnShield-DNN is the investigation of an integrated, resource-aware methodology that connects:

Fault Injection

- Adaptive Channel-Level Vulnerability Discovery
- Vulnerability Ranking
- Cross-Layer Fault Interaction Analysis
- Targeted Fault-Aware Protection.

The research therefore positions its novelty at the level of system integration, experimental formulation, and cross-layer interaction analysis rather than claiming that TD3, fault injection, vulnerability ranking, or fault-aware training individually constitute new techniques.

The proposed work will establish the validity and significance of these contributions through controlled experiments and comparison with appropriate baseline methods. Any claim of superiority or quantitative improvement will be made only after the corresponding experimental results have been obtained.

4. PROPOSED VULNSHIELD-DNN FRAMEWORK

4.1 Overview of the Proposed Framework

VulnShield-DNN is a software-based framework designed to identify fault-sensitive internal channels of Deep Neural Networks (DNNs) and use the resulting vulnerability information to guide targeted robustness enhancement. The framework is developed around the observation that the effect of an internal fault is not uniform across a DNN: faults affecting different channels can produce substantially different levels of prediction degradation. The proposed methodology therefore treats vulnerability identification as a selective search problem rather than requiring exhaustive evaluation of every possible channel.

The framework operates on trained convolutional neural networks and uses controlled software-based fault injection to evaluate channel-level sensitivity. The primary fault model defined for the project is a channel stuck-at-zero fault. During an evaluation, the output of a selected convolutional channel is temporarily replaced with zero using a PyTorch forward hook. This approach allows the effect of the simulated fault to be measured without permanently modifying the original DNN architecture or its tensor dimensions.

The vulnerability-discovery stage is formulated as an adaptive search problem. A Twin Delayed Deep Deterministic Policy Gradient (TD3) agent selects candidate channels for fault injection based on feedback obtained from previous evaluations. The objective is to identify channels that produce significant model degradation while operating under a predefined fault-injection budget. The resulting observations are used to construct a ranked representation of channel vulnerability.

TD3 is not used in isolation. The framework includes several comparison strategies to establish a common experimental baseline. These include random channel selection, activation-based Top-K ranking, gradient or Taylor-based ranking, and a layer-wise Deep Deterministic Policy Gradient (DDPG) approach. The methods are evaluated under the same experimental conditions so that their vulnerability-discovery efficiency and resulting rankings can be compared.

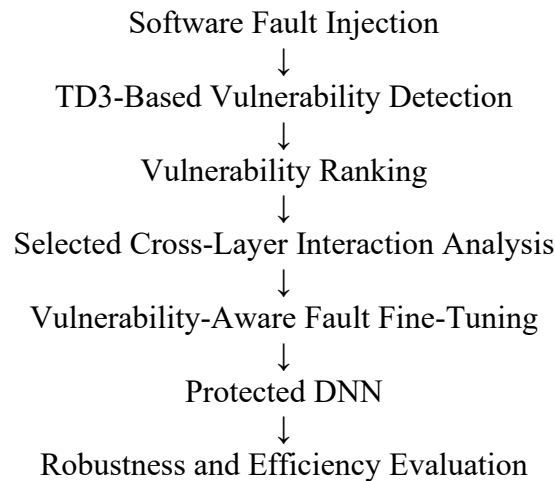
Following vulnerability discovery, the framework performs a selected cross-layer interaction analysis. Highly ranked channels belonging to different network layers or adjacent network blocks are evaluated under simultaneous fault conditions. Their combined degradation is compared with the degradation observed when the corresponding channels are faulted individually. The purpose is to determine whether simultaneous cross-layer faults exhibit interaction effects, including amplification or masking, that cannot be inferred from independent vulnerability measurements.

The framework also includes a supporting comparison between hardware-fault sensitivity and adversarial sensitivity. Fault-sensitive channels identified through the software fault-injection process are compared with channel responses observed under FGSM and PGD adversarial inputs. The purpose is to determine whether the two perturbation mechanisms identify overlapping or distinct channel-level sensitivity patterns. This analysis remains complementary to the primary hardware-fault reliability investigation.

The final stage converts vulnerability information into targeted protection. Instead of applying fault-aware fine-tuning uniformly across the complete network, channels selected from the vulnerability ranking are exposed to the defined fault condition during fine-tuning. The training objective combines clean-data classification loss with faulted-data classification loss so that the model is encouraged to retain normal predictive performance while improving its tolerance to the simulated channel fault.

The protection stage is evaluated under predefined protection budgets of 1%, 3%, 5%, and 10%. Different channel-selection strategies are compared under equal budgets, including random, heuristic, DDPG-guided, and TD3-guided selection. This enables the study to evaluate the relationship between the amount of protection applied and the resulting robustness improvement, while also monitoring the effect on clean-data accuracy and computational overhead.

The complete framework can therefore be represented as the following sequence:



The framework will be evaluated primarily using ResNet-18 and additionally using VGG-16 on the CIFAR-10 dataset. The evaluation will consider clean accuracy, faulted accuracy, vulnerability-discovery efficiency, robustness improvement, computational overhead, selected cross-layer fault combinations, and unseen channel faults. Limited weight bit-flip testing is also included as an additional validation condition.

The proposed framework is intended as an academic research prototype for software-based DNN reliability assessment and targeted model hardening. It does not claim to replace physical hardware fault-injection experiments. Hardware-specific validation, physical fault models, larger application-specific datasets, and deployment certification would be required before applying the methodology to safety-critical production systems.

4.2 Design Principles

The design of VulnShield-DNN is based on a set of principles intended to support systematic vulnerability assessment, targeted robustness enhancement, and controlled experimental evaluation of Deep Neural Networks under simulated hardware-induced faults.

1. Channel-Level Vulnerability Assessment

The framework operates at the internal channel level rather than treating the complete DNN as a single unit. This allows the effect of faults affecting individual convolutional channels to be measured and compared. The channel-level representation provides the basis for vulnerability ranking and subsequent protection selection.

2. Controlled Software Fault Injection

Fault injection is performed through a controlled software mechanism so that individual fault conditions can be reproduced consistently across experiments. The defined primary fault model is a channel stuck-at-zero fault, where the selected channel output is temporarily replaced with zero during model execution.

3. Adaptive Vulnerability Discovery

The framework treats fault-location selection as an adaptive search problem. Instead of evaluating all possible channels with equal priority, the TD3 agent uses feedback from previous fault-injection evaluations to guide subsequent channel selection. This design is intended to reduce unnecessary fault-injection evaluations under a limited search budget.

4. Baseline-Based Comparative Evaluation

The effectiveness of TD3-based vulnerability discovery is evaluated against alternative selection strategies rather than in isolation. The proposed comparison includes random selection, activation-based Top-K ranking, gradient/Taylor-based ranking, and layer-wise DDPG. The comparison is performed under controlled experimental conditions to distinguish the effect of the vulnerability-selection strategy from other experimental factors.

5. Vulnerability-Guided Channel Prioritization

The framework converts the observations obtained during fault injection into a vulnerability ranking. Channels exhibiting greater measured impact on model performance receive higher priority for subsequent analysis and protection. This ranking provides a common interface between the vulnerability-discovery and robustness-enhancement stages.

6. Explicit Cross-Layer Interaction Analysis

The framework does not assume that individual channel vulnerability completely describes simultaneous fault behaviour. Selected vulnerable channels from different network layers are therefore evaluated under simultaneous fault conditions. Their combined effect is compared with the corresponding individual fault effects to investigate possible amplification, masking, or other interaction behaviour.

7. Budget-Aware Protection

Protection resources are treated as constrained rather than unlimited. Only a predefined proportion of network channels is selected for targeted fault-aware fine-tuning. The project evaluates protection budgets of 1%, 3%, 5%, and 10%, allowing the relationship between protection coverage and robustness improvement to be studied.

8. Preservation of Clean-Model Performance

Robustness improvement is not evaluated solely through faulted accuracy. Clean-data accuracy is also monitored because excessive fault-aware fine-tuning could alter the model's normal predictive behaviour. The protection objective therefore considers both faulted and clean performance.

9. Separation of Discovery and Protection

Vulnerability discovery and robustness enhancement are treated as distinct stages. The first stage identifies and ranks fault-sensitive channels, while the second stage uses selected channels for targeted fault-aware fine-tuning. This separation enables the effectiveness of the vulnerability ranking to be evaluated independently and through its downstream protection performance.

10. Unseen-Fault Validation

The framework includes evaluation on channel faults that were not used during the vulnerability-discovery or protection process. This is intended to determine whether the protected model improves its robustness beyond only the specific fault conditions encountered during training.

11. Cross-Architecture Evaluation

The experimental design considers ResNet-18 as the primary architecture and VGG-16 as an additional architecture. This provides a basis for determining whether the observed behaviour is specific to one network architecture or can also be observed across different convolutional DNN structures.

12. Reproducible Experimental Evaluation

The framework uses fixed datasets, defined fault models, controlled fault-injection procedures, specified search and protection budgets, and explicit evaluation metrics. These controls are intended to make comparisons between vulnerability-discovery and protection strategies reproducible and interpretable.

13. Software-Level Research Scope

The proposed framework is designed as a software-based research methodology. It evaluates simulated faults within DNN execution and does not claim to reproduce every physical characteristic of hardware faults. Physical hardware validation would constitute a separate stage beyond the scope of the present work.

Collectively, these design principles establish VulnShield-DNN as a controlled, vulnerability-guided, and resource-aware framework in which fault injection, adaptive vulnerability discovery, cross-layer interaction analysis, and targeted robustness enhancement are evaluated as connected but distinguishable research stages.

4.3 Overall System Architecture

The VulnShield-DNN framework is organized as a sequential reliability-analysis and protection pipeline. The architecture connects controlled software fault injection, adaptive vulnerability discovery, vulnerability ranking, cross-layer interaction analysis, vulnerability-aware fine-tuning, and final robustness evaluation.

The overall architecture consists of the following major stages:

1. DNN Model and Dataset

A trained convolutional Deep Neural Network is provided as the target model. The primary experimental setting uses ResNet-18, with VGG-16 considered as an additional architecture, and CIFAR-10 as the primary dataset. The original trained model is retained as the clean reference model for subsequent comparisons.

2. Software Fault Injection Layer

The fault-injection layer introduces controlled faults into selected internal channels during model execution. The primary fault model is a channel stuck-at-zero fault, implemented by temporarily replacing the output of a selected channel with zero. The injection mechanism preserves the original model structure and allows individual fault conditions to be evaluated repeatedly.

3. Vulnerability Discovery Layer

The vulnerability-discovery layer determines which channels should be evaluated for fault sensitivity. The principal method is a TD3-based reinforcement-learning agent. The agent selects candidate channels, receives feedback from the resulting fault-injection experiment, and updates its search behaviour based on the observed model degradation.

4. Vulnerability Evaluation and Ranking Layer

The results obtained from fault-injection experiments are converted into channel-level vulnerability information. Channels are ranked according to their measured impact on model performance. This ranking provides the basis for selecting high-impact channels for subsequent cross-layer analysis and targeted protection.

5. Comparative Vulnerability-Selection Layer

To evaluate the effectiveness of the TD3-based approach, the architecture also supports alternative vulnerability-selection strategies. These include random selection, activation-based Top-K ranking, gradient/Taylor-based ranking, and layer-wise DDPG. The same experimental environment is used to compare the resulting vulnerability-discovery behaviour.

6. Cross-Layer Interaction Analysis Layer

Selected highly vulnerable channels from different network layers are combined into simultaneous fault scenarios. The model is evaluated under these combined faults, and the resulting degradation is compared with the degradation produced by the corresponding individual faults. This stage is used to investigate cross-layer interaction, including possible amplification or masking effects.

7. Vulnerability-Aware Protection Layer

The vulnerability ranking is then used to select channels for targeted fault-aware fine-tuning. Rather than modifying the complete network uniformly, the protection stage concentrates the additional training on selected vulnerable channels.

8. Budget Controller

The protection stage operates under predefined protection budgets. The experimental configuration considers 1%, 3%, 5%, and 10% protection budgets. The budget determines the maximum proportion of channels that can be selected for targeted protection.

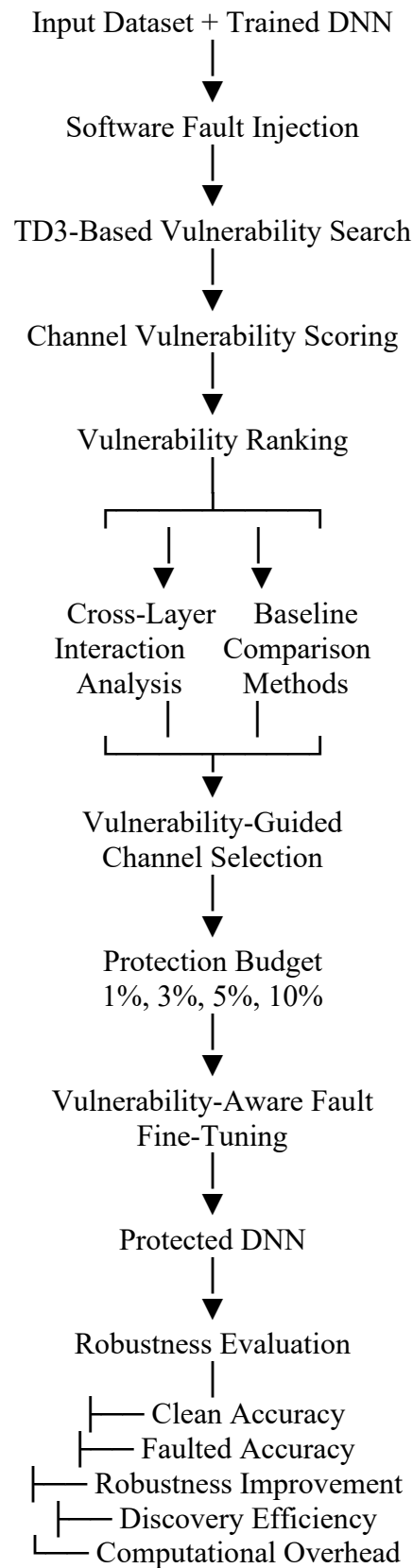
9. Protected DNN

After vulnerability-aware fine-tuning, the resulting model is treated as the protected DNN. The protected model is evaluated under both clean and faulted conditions to determine whether the targeted protection improves fault tolerance without unacceptable degradation of normal predictive performance.

10. Robustness Evaluation

The final evaluation measures clean accuracy, faulted accuracy, robustness improvement, vulnerability-discovery efficiency, computational overhead, and performance under selected unseen faults. These measurements are used to compare the proposed approach with the baseline vulnerability-selection and protection strategies.

The overall architecture can therefore be represented as:



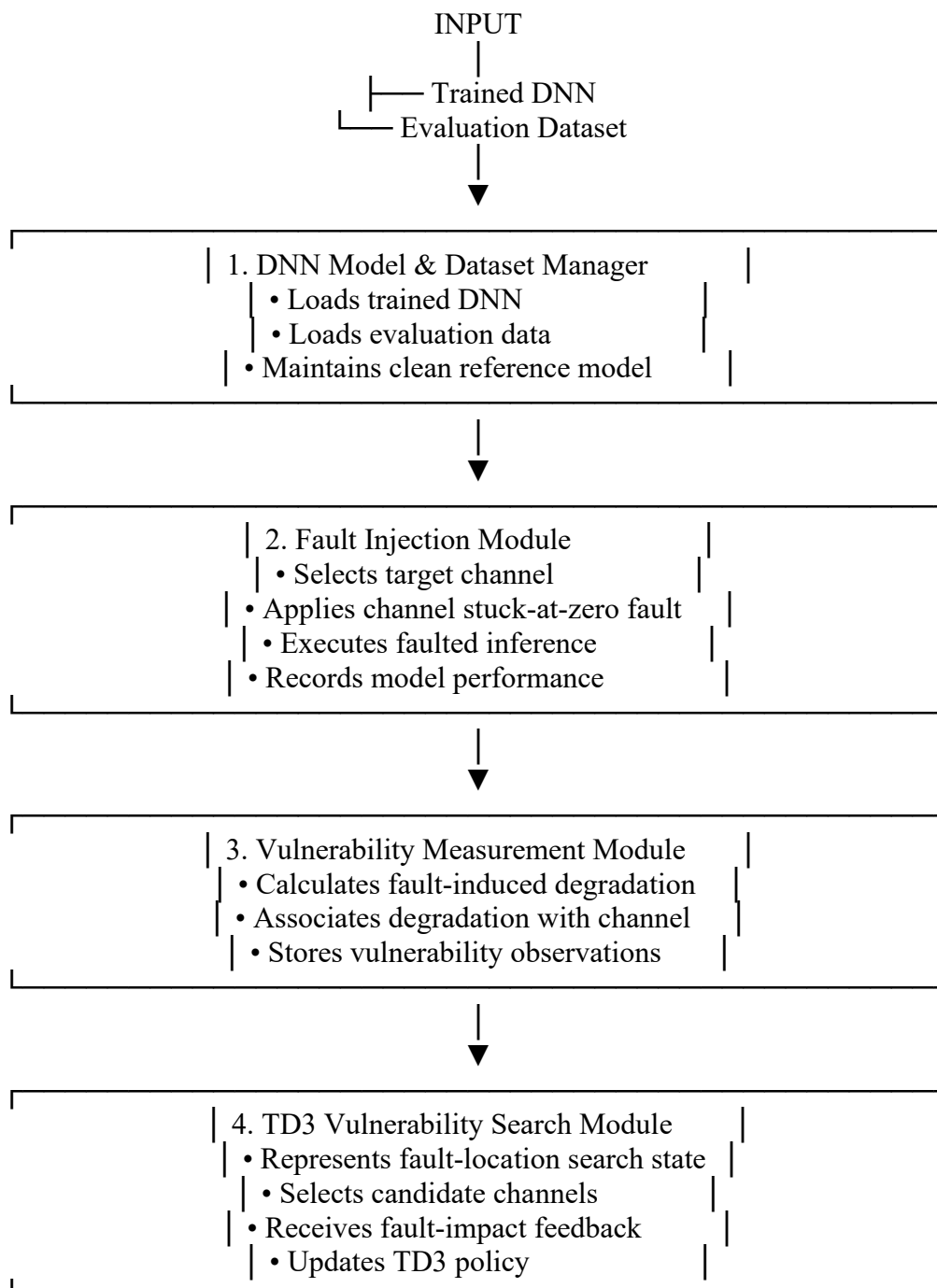
The architecture separates vulnerability discovery, interaction analysis, and protection into distinct functional stages while maintaining information flow between them. The vulnerability ranking acts as the principal decision interface between the discovery stage and the subsequent cross-layer analysis and protection stages.

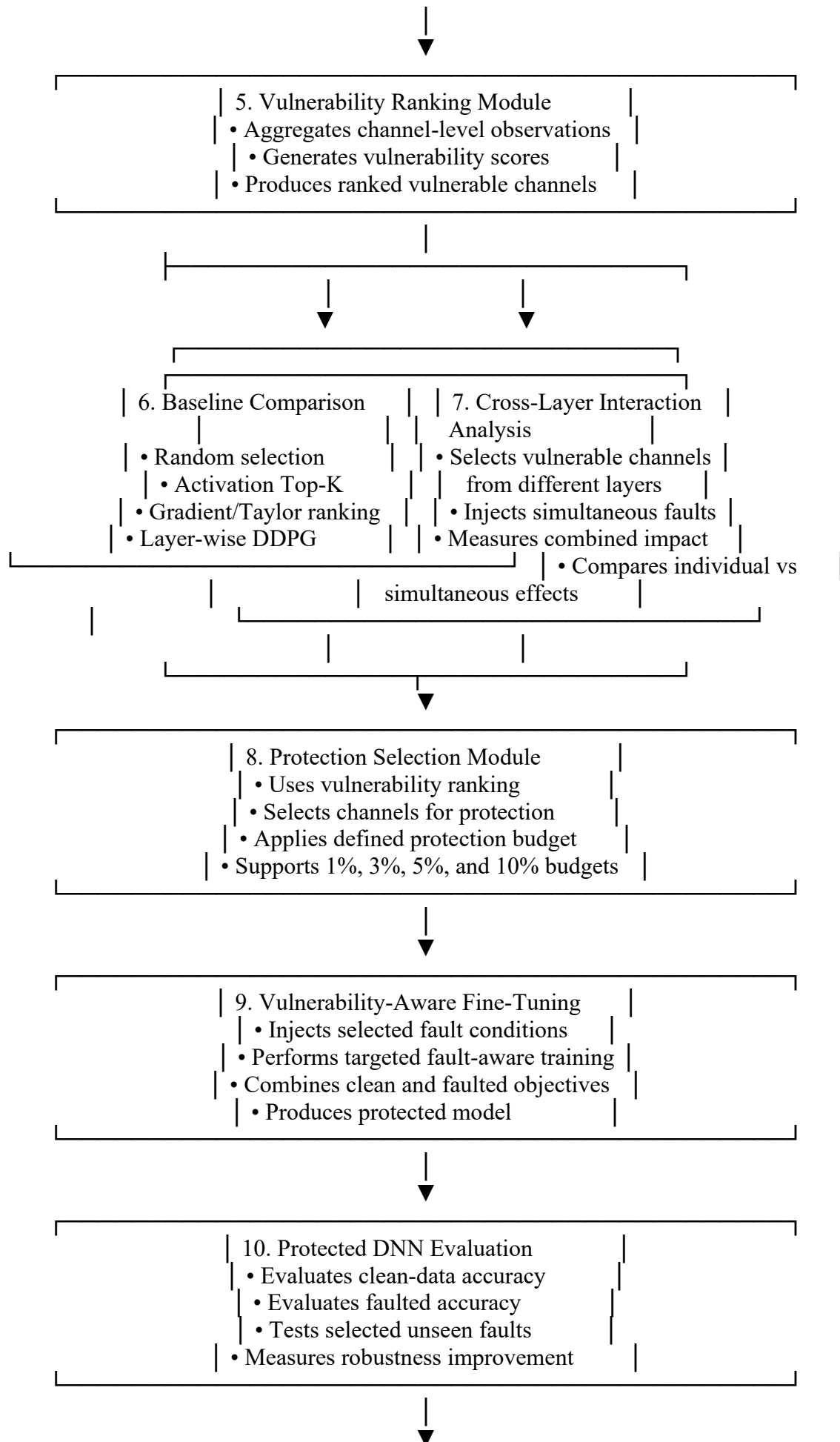
This architecture is intended to provide a controlled experimental framework for investigating whether adaptive vulnerability discovery can improve the efficiency of fault analysis and whether the resulting vulnerability information can be effectively used for targeted robustness enhancement under constrained protection resources.

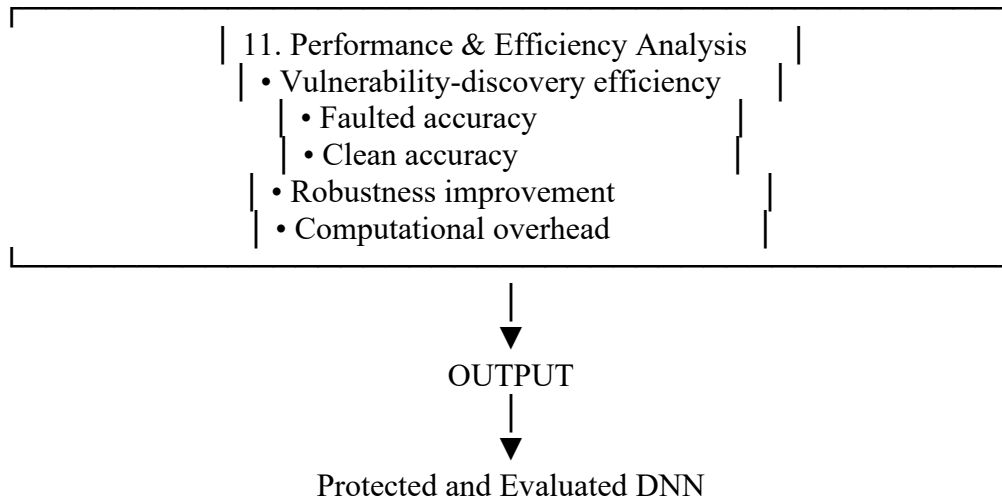
4.4 Functional Block Diagram

The functional architecture of VulnShield-DNN represents the major computational and decision-making modules required to perform vulnerability assessment and targeted robustness enhancement. Each module performs a specific function and passes its output to the subsequent stage.

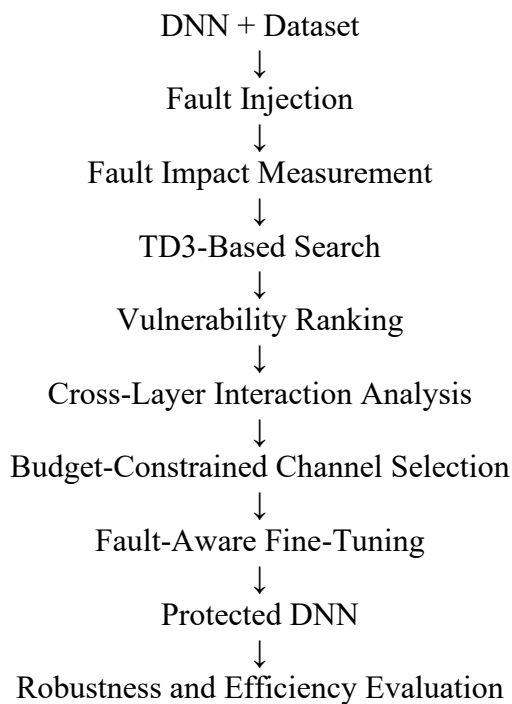
The functional blocks are organized as follows:







The functional relationship between the principal modules can be summarized as:



The architecture also maintains a comparison path throughout the vulnerability-discovery stage. Random selection, activation-based Top-K ranking, gradient/Taylor-based ranking, and layer-wise DDPG are evaluated as baseline approaches against the TD3-based method. This enables the contribution of the adaptive search mechanism to be assessed independently of the subsequent protection process.

The functional design therefore separates three principal responsibilities:

1. Vulnerability Discovery — identify and rank channels that demonstrate high fault sensitivity.
2. Interaction Analysis — investigate whether selected vulnerable channels exhibit different behaviour when faults occur simultaneously across network layers.

3. Protection — use the vulnerability information to allocate limited fault-aware fine-tuning resources toward selected channels.

The final output of the architecture is not simply a vulnerability list. The framework produces a protected DNN together with experimental measurements describing the effectiveness and computational requirements of the vulnerability-discovery and protection processes.

4.5 End-to-End Framework Workflow

The VulnShield-DNN workflow defines the complete sequence through which a trained Deep Neural Network is analysed for fault vulnerability, vulnerable channels are identified and ranked, selected cross-layer fault interactions are examined, and limited protection resources are allocated for targeted robustness enhancement.

The workflow consists of the following stages:

Step 1: Load the Trained DNN and Dataset

A trained convolutional DNN and the evaluation dataset are loaded into the experimental environment. The clean model is evaluated first to establish the baseline performance against which all subsequent faulted and protected-model results are compared.

Step 2: Establish Clean-Model Baseline

The clean model is evaluated without fault injection. Clean-data accuracy is recorded as the reference performance. This baseline is retained throughout the experiment to quantify any degradation caused by fault injection and any changes introduced through fault-aware fine-tuning.

Step 3: Configure the Fault Model

The software fault-injection mechanism is configured using the defined channel-level stuck-at-zero fault model. A selected channel's output is temporarily replaced with zero during model execution while the original network architecture and tensor dimensions remain unchanged.

Step 4: Perform Initial Fault-Injection Evaluation

Candidate channels are subjected to controlled fault injection. The resulting model performance is measured and compared with the clean-model baseline. The observed degradation forms the feedback used by the vulnerability-discovery process.

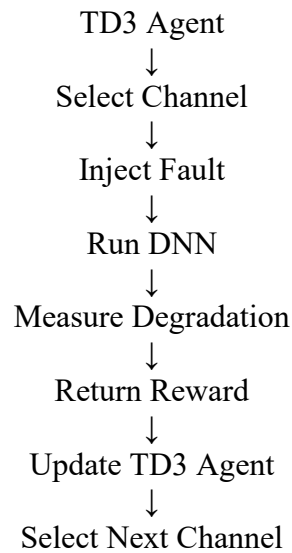
Step 5: Initialize Vulnerability Search

The TD3 agent is initialized with the defined state representation, action space, reward formulation, and search budget. The agent selects candidate channels for subsequent fault-injection evaluation.

Step 6: Execute TD3-Guided Fault Search

For each search step, TD3 selects a candidate channel. The fault-injection module applies the defined fault to that channel, executes inference, and returns the measured fault impact to the RL environment.

The sequence is:



This process continues until the predefined fault-injection budget is reached.

Step 7: Generate Vulnerability Ranking

The observations collected during fault injection are aggregated to calculate channel-level vulnerability information. Channels are ranked according to their measured fault impact, producing a prioritized vulnerability list.

Step 8: Perform Baseline Comparison

The same vulnerability-discovery setting is evaluated using alternative approaches:

- Random channel selection
- Activation-based Top-K ranking
- Gradient/Taylor-based ranking
- Layer-wise DDPG

The purpose is to determine whether the TD3-based strategy provides an advantage in vulnerability discovery efficiency and identification of high-impact channels.

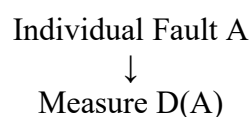
Step 9: Select Cross-Layer Vulnerable Channels

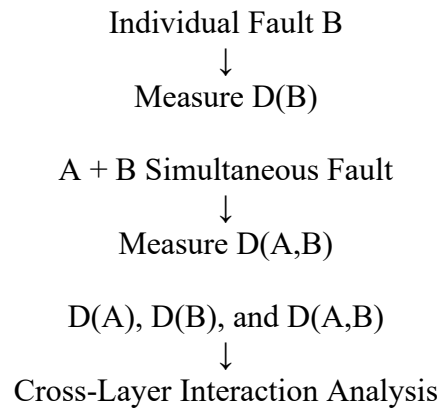
Highly ranked channels from different network layers are selected for interaction analysis. The selection is restricted to a manageable number of combinations to avoid exhaustive pairwise or higher-order fault injection.

Step 10: Inject Simultaneous Cross-Layer Faults

The selected channels are faulted simultaneously. The resulting degradation is measured and compared with the degradation produced by the corresponding individual faults.

The analysis follows:





The comparison is used to determine whether the combined fault exhibits behaviour such as masking, amplification, or another measurable interaction effect.

Step 11: Determine Protection Budget

A predefined protection budget is established. The project considers protection levels of 1%, 3%, 5%, and 10%. The budget determines the proportion of channels that can receive targeted fault-aware fine-tuning.

Step 12: Select Channels for Protection

Channels are selected according to the vulnerability-ranking strategy being evaluated. TD3-ranked channels are compared with channels selected using random, heuristic, and DDPG-based strategies under equivalent protection budgets.

Step 13: Perform Vulnerability-Aware Fault Fine-Tuning

The selected channels are subjected to the defined fault condition during targeted fine-tuning. The training process combines clean-data and faulted-data objectives so that the model is encouraged to maintain normal predictive behaviour while improving tolerance to the simulated fault.

Step 14: Generate the Protected DNN

After the defined fine-tuning process, the resulting model is retained as the protected DNN. The original clean model is preserved separately to enable direct comparison between the unprotected and protected models.

Step 15: Evaluate the Protected Model

The protected model is evaluated under multiple conditions:

- Clean evaluation data
- Known fault conditions
- Selected unseen channel faults
- Selected simultaneous cross-layer faults
- Additional limited weight bit-flip conditions where applicable

The purpose is to determine whether the protection provides robustness beyond the exact fault conditions used during fine-tuning.

Step 16: Perform Adversarial-Sensitivity Comparison

As a supporting analysis, FGSM and PGD adversarial examples are used to evaluate channel-level sensitivity to input perturbations. The resulting sensitivity information is compared with the hardware-fault vulnerability ranking to identify overlapping or distinct sensitivity patterns.

Step 17: Calculate Evaluation Metrics

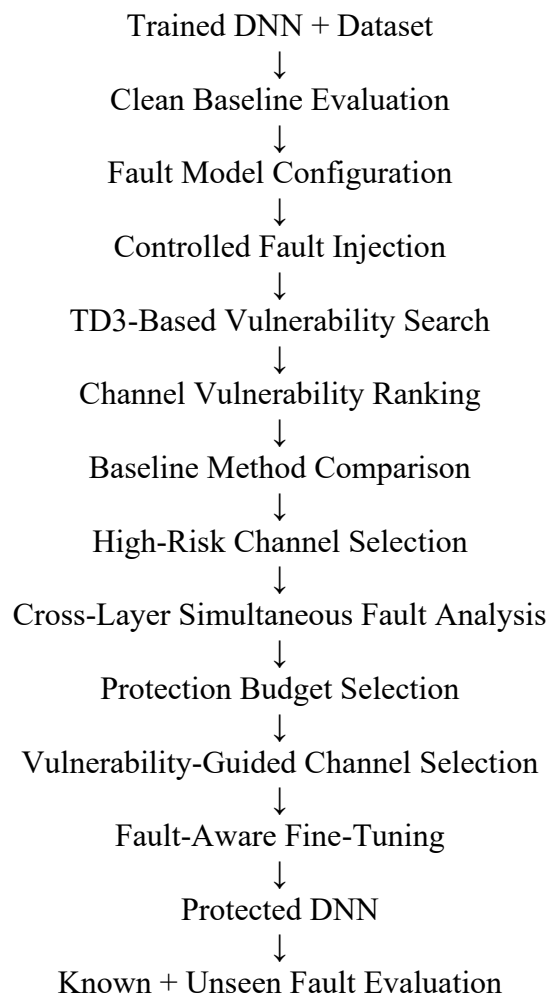
The final analysis considers:

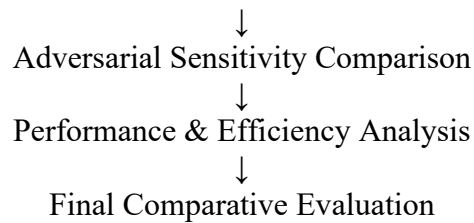
- Clean accuracy
- Faulted accuracy
- Accuracy degradation
- Robustness improvement
- Vulnerability-discovery efficiency
- Number of fault-injection evaluations
- Protection coverage
- Computational overhead

Step 18: Compare All Approaches

The final results are compared across TD3, DDPG, random selection, activation-based ranking, and gradient/Taylor-based ranking. The comparison is performed across different fault-injection and protection budgets.

The complete workflow can be summarized as:





The workflow maintains a clear separation between discovery, analysis, and protection. The vulnerability-discovery stage determines where significant fault sensitivity is observed; the cross-layer stage investigates whether selected vulnerable channels exhibit interaction under simultaneous faults; and the protection stage evaluates whether the vulnerability information can be converted into measurable robustness improvement under constrained resources.

This workflow provides the operational basis for the individual modules described in the subsequent sections of the proposed framework.

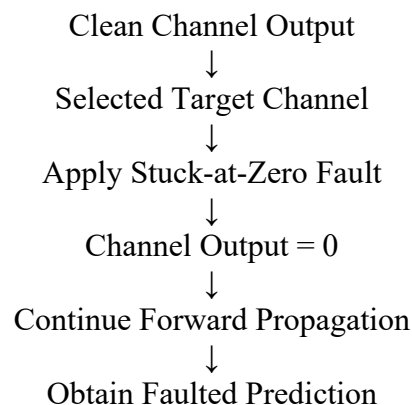
4.6 Fault Injection Module

The Fault Injection Module provides the controlled experimental mechanism used to evaluate the sensitivity of internal DNN channels to simulated hardware-induced faults. It forms the foundation of the VulnShield-DNN framework because the vulnerability-discovery process obtains its feedback directly from the model's response to injected faults.

The module operates at the internal channel level of convolutional layers. A channel is treated as a selected computational feature map produced by a convolutional operation. Instead of modifying the trained network permanently, the proposed implementation temporarily alters the output of the selected channel during inference.

The primary fault model considered in the proposed work is the channel stuck-at-zero fault. Under this fault condition, the output associated with a selected channel is replaced by zero during the affected inference operation. The remaining channels and the network architecture remain unchanged.

The fault injection process can be represented as:



The implementation uses a PyTorch forward hook to intercept the output of the selected convolutional layer during model execution. The hook identifies the target channel and replaces its output with a zero-valued tensor of the corresponding spatial dimensions. Once the forward pass is completed, the temporary intervention is removed so that subsequent evaluations can be performed independently.

The use of a runtime hook provides three important properties:

1. The original DNN architecture is preserved.
2. Individual channels can be evaluated without permanently modifying model parameters.
3. The same trained model can be repeatedly evaluated under different fault locations.

For a convolutional layer with output tensor:

$$X \in \mathbb{R}^{(B \times C \times H \times W)}$$

where:

B = batch size,
C = number of output channels,
H = spatial height, and
W = spatial width,

a fault affecting channel c can be represented as:

$$X[:, c, :, :] = 0$$

while all other channels remain unchanged:

$$X[:, j, :, :] = X[:, j, :, :] \text{ for } j \neq c$$

The faulted tensor X' is then passed to the subsequent layers of the network.

The effect of the injected fault is measured relative to the clean-model result. If A_{clean} represents the accuracy of the original model and A_{fault} represents the accuracy after injecting a fault into channel c, the basic accuracy degradation can be represented as:

$$\Delta A(c) = A_{\text{clean}} - A_{\text{fault}}(c)$$

A larger positive $\Delta A(c)$ indicates greater degradation under the selected fault condition and therefore provides stronger evidence that the evaluated channel is fault-sensitive under the experimental configuration.

The same principle can be applied to the loss-based evaluation. If L_{clean} denotes the clean-model loss and $L_{\text{fault}}(c)$ denotes the loss obtained after faulting channel c, the change in loss can be expressed as:

$$\Delta L(c) = L_{\text{fault}}(c) - L_{\text{clean}}$$

Both accuracy degradation and loss increase can provide information about the effect of the injected fault. The specific metric used for reinforcement-learning feedback and vulnerability ranking must remain consistent across the compared methods.

The module records the following information for each fault-injection experiment:

- Network architecture
- Layer identifier
- Channel identifier
- Fault type

- Clean-model accuracy
- Faulted-model accuracy
- Accuracy degradation
- Clean loss
- Faulted loss
- Fault-injection execution information
- Computational time, where measured

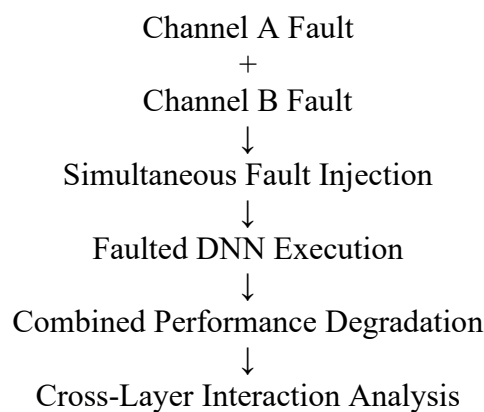
These observations form the input to the subsequent Vulnerability Measurement and TD3-Based Vulnerability Search modules.

The Fault Injection Module also supports two different experimental purposes.

First, individual fault injection is used during vulnerability discovery. The objective is to determine the effect of faults affecting individual channels and generate the observations required for channel-level vulnerability ranking.

Second, simultaneous fault injection is used during cross-layer interaction analysis. In this case, two or more selected channels from different network layers are faulted during the same inference process. The resulting degradation is compared with the corresponding individual fault effects.

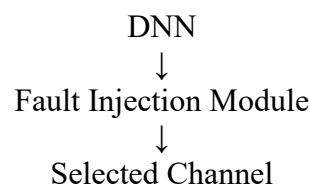
The simultaneous-fault process can therefore be represented as:

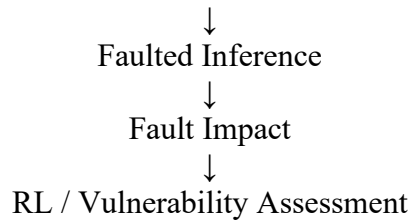


To maintain experimental validity, fault injection is separated from the normal clean-model evaluation. The clean reference model is evaluated before fault injection, and the resulting performance is retained as the baseline. Each fault experiment is then compared against this reference.

The module is also designed to support controlled fault-injection budgets. During vulnerability discovery, the number of channels evaluated can be restricted to a predefined search budget. This allows the efficiency of TD3-based selection to be compared against random and other baseline strategies without requiring exhaustive evaluation of every channel.

The module therefore acts as the experimental interface between the DNN and the vulnerability-discovery algorithm:





The fault-injection mechanism is intentionally implemented at the software level. Consequently, the injected stuck-at-zero condition represents a controlled simulation of a fault effect rather than a physical reproduction of a particular hardware failure mechanism. The results therefore characterize model sensitivity under the defined software fault model and should not be interpreted as direct measurements of physical hardware failure rates.

The primary output of the Fault Injection Module is a collection of reproducible fault-impact observations associated with individual DNN channels. These observations are subsequently used by the Vulnerability Measurement Module to quantify channel sensitivity and by the TD3 agent to guide the adaptive vulnerability-discovery process.

4.7 Channel Vulnerability Representation

The Channel Vulnerability Representation defines how the fault sensitivity of individual DNN channels is represented, measured, and provided to the vulnerability-discovery and protection stages of VulnShield-DNN. Since the proposed framework operates at the channel level, a consistent representation is required to associate each fault-injection experiment with a specific network location and its measured effect on model performance.

For a convolutional layer l containing C_l output channels, an individual channel can be represented by the pair:

(l, c)

where:

l = identifier of the convolutional layer

c = identifier of the output channel within that layer.

The complete set of candidate channels in the network can therefore be represented as:

$$C = \{(l, c) \mid l \in L, c \in C_l\}$$

where L represents the set of convolutional layers considered for fault analysis.

For every selected channel, the fault-injection module produces an observation containing the channel location, fault condition, and resulting model behaviour. A basic channel-level observation can be represented as:

$$O(l, c) = \{l, c, f, A_{\text{clean}}, A_{\text{fault}}, \Delta A\}$$

where:

l = layer identifier,

c = channel identifier,

f = applied fault condition,

A_{clean} = clean-model accuracy,
 A_{fault} = accuracy under the injected fault, and
 ΔA = observed accuracy degradation.

The primary degradation measure is defined as:

$$\Delta A(l,c) = A_{\text{clean}} - A_{\text{fault}}(l,c)$$

A larger value of $\Delta A(l,c)$ indicates that the corresponding channel produced greater accuracy degradation under the evaluated fault condition.

The vulnerability representation can consequently be expressed as a channel-wise vulnerability vector:

$$V = [v_1, v_2, \dots, v_N]$$

where N is the number of evaluated channels and v_i represents the vulnerability value associated with the corresponding channel.

For a channel indexed by i :

$$v_i = \Delta A_i$$

when accuracy degradation is used as the primary vulnerability measure.

The resulting representation allows the evaluated channels to be ordered according to their observed fault impact:

$$v_1 \geq v_2 \geq \dots \geq v_N$$

where the first-ranked channels exhibit the largest measured degradation under the defined fault model.

This ranking is used by subsequent stages to identify high-priority channels for further analysis and protection.

Channel-Level State Representation for TD3

For reinforcement-learning-based vulnerability discovery, the channel information must additionally be represented as an environment state. The state contains information available to the agent at a particular stage of the vulnerability-search process.

A conceptual state representation is:

$$s_t = [\text{network location, channel information, previous fault observations, search information}]$$

where t represents the current search step.

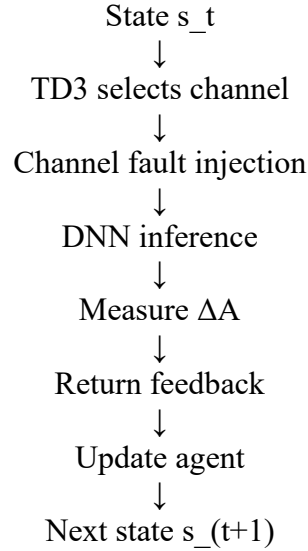
The state representation allows the TD3 agent to distinguish between candidate fault locations and incorporate information obtained from previous fault-injection experiments.

The exact state encoding is maintained consistently across the TD3 experiments so that changes in performance can be attributed to the vulnerability-selection strategy rather than changes in the representation.

Fault-Impact Feedback

After the TD3 agent selects a channel, the Fault Injection Module evaluates that channel and returns a fault-impact measurement. This measurement provides the feedback required by the reinforcement-learning environment.

The interaction can be represented as:



The feedback therefore links the internal channel representation with the observed behaviour of the complete DNN.

Vulnerability Ranking Representation

After sufficient fault-injection observations have been collected, the framework produces a ranked channel list:

$$R = [(l_1, c_1, v_1), (l_2, c_2, v_2), \dots, (l_k, c_k, v_k)]$$

where each entry contains:

- Layer identifier
- Channel identifier
- Vulnerability value

The ranked representation enables the framework to select the top-K vulnerable channels:

$$\text{Top-K}(R) = \{r_1, r_2, \dots, r_K\}$$

These selected channels can subsequently be used for cross-layer interaction analysis and vulnerability-aware protection.

Cross-Layer Representation

For cross-layer analysis, the vulnerability representation additionally retains the network-layer identity of each selected channel. This is necessary because the interaction analysis specifically considers channels originating from different network layers.

A selected pair can therefore be represented as:

$$P = ((l_i, c_i), (l_j, c_j))$$

subject to:

$$l_i \neq l_j$$

The corresponding individual and simultaneous fault measurements are then retained as:

D_i = individual degradation of channel i

D_j = individual degradation of channel j

D_{ij} = degradation when channels i and j are faulted simultaneously.

These measurements provide the basis for evaluating whether the simultaneous fault produces behaviour different from the individual fault observations.

Protection Representation

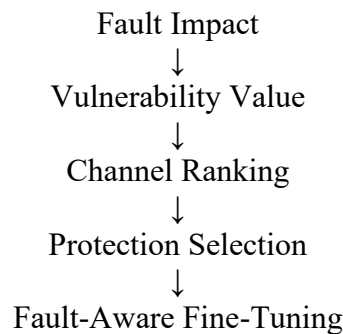
The vulnerability representation also provides the input for budget-aware protection. Given a protection budget B and a ranked channel set R , the framework selects a limited subset of channels:

$$P_B \subseteq R$$

where the size of P_B is constrained by the predefined protection budget.

For example, the project evaluates protection budgets of 1%, 3%, 5%, and 10%. The selected channels are then used as targets for vulnerability-aware fault fine-tuning.

This creates a direct relationship between vulnerability score and protection priority:



Representation Requirements

The channel vulnerability representation is designed to satisfy four requirements:

1. Location Awareness

Every vulnerability observation must identify the network layer and channel to which the fault was applied.

2. Fault-Impact Association

The representation must associate each channel with the measured effect of the injected fault.

3. Ranking Compatibility

The representation must support ordering channels according to their observed vulnerability.

4. Protection Compatibility

The resulting ranking must be usable for selecting a limited number of channels for subsequent fault-aware fine-tuning.

Thus, the channel vulnerability representation acts as the information bridge between the fault-injection mechanism, TD3-based vulnerability discovery, cross-layer interaction analysis, and budget-aware protection stages.

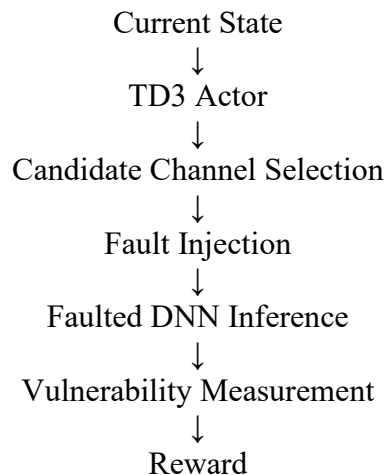
The final output of this module is a structured channel-level vulnerability representation that can be used to identify high-impact channels and provide consistent input to the subsequent TD3-based vulnerability-detection module.

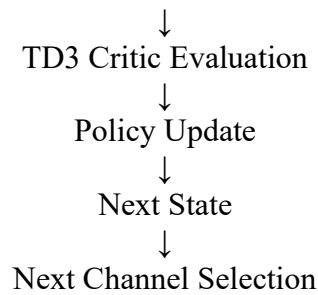
4.8 TD3-Based Vulnerability Detection

The TD3-Based Vulnerability Detection module formulates internal DNN fault-location discovery as an adaptive decision-making problem. The objective of this module is to identify fault-sensitive channels efficiently under a predefined fault-injection budget rather than evaluating every candidate channel with equal priority.

Twin Delayed Deep Deterministic Policy Gradient (TD3) is selected as the principal reinforcement-learning method. TD3 is an actor-critic algorithm designed for continuous action spaces and incorporates mechanisms intended to reduce overestimation and improve learning stability. Within VulnShield-DNN, TD3 is used as the adaptive search mechanism for selecting candidate channel locations for fault injection.

The TD3 module interacts with the Fault Injection Module through an iterative feedback loop:





4.8.1 Reinforcement-Learning Formulation

The vulnerability-discovery problem is represented as a Markov Decision Process (MDP). At each step t , the agent observes a state s_t , selects an action a_t corresponding to a candidate channel, receives a reward r_t based on the measured fault impact, and transitions to a new state s_{t+1} .

The interaction can be expressed as:

$$(s_t, a_t) \rightarrow \text{Fault Injection} \rightarrow r_t, s_{t+1}$$

where:

s_t = current vulnerability-search state

a_t = selected fault location

r_t = reward derived from observed fault impact

s_{t+1} = updated search state.

The objective of the agent is to learn a policy that identifies high-impact channels while operating within the specified number of allowed fault-injection evaluations.

4.8.2 State Representation

The state represents the information available to the TD3 agent during vulnerability discovery. It contains the channel-search information and the vulnerability observations accumulated during previous fault-injection evaluations.

A conceptual representation is:

s_t = [channel information, layer information, previous vulnerability observations, search information]

The state is updated after each selected channel is evaluated. The representation is maintained consistently throughout the TD3 experiments to ensure that comparisons with the baseline methods remain meaningful.

4.8.3 Action Representation

The action corresponds to the candidate fault location selected for evaluation.

Conceptually:

$$a_t \rightarrow (\text{layer } l, \text{channel } c)$$

The selected channel is passed to the Fault Injection Module, which applies the defined stuck-at-zero fault and evaluates the resulting model behaviour.

Because TD3 is designed for continuous action spaces while channel selection is inherently discrete, the framework defines an explicit continuous-to-discrete mapping rather than assuming the two are directly compatible. The TD3 actor first produces a continuous candidate-location vector $a_t \in \mathbb{R}^d$, where d is the dimensionality of the candidate-location embedding rather than the number of channels directly. This continuous output is then converted into a valid discrete channel index through a fixed nearest-neighbour assignment: each eligible (layer, channel) pair is associated with a corresponding point in the same embedding space, and the channel whose embedding is closest to a_t , measured using Euclidean distance, is selected as the discrete action for that step. Where multiple layers are involved, the layer identifier is resolved first from a designated sub-range of a_t , after which the channel index within that layer is resolved from the remaining components, so that the mapping decomposes as: TD3 continuous action $\rightarrow$ candidate-location embedding $\rightarrow$ nearest valid (layer, channel) pair $\rightarrow$ discrete channel index $\rightarrow$ Fault Injection Module. This mapping is defined once and kept fixed throughout all experiments, ensuring that TD3 is compared against the alternative vulnerability-discovery strategies on the basis of which channels it selects rather than on differences in how the discrete selection is derived. The full mapping specification, including the construction of the candidate-location embedding, is given in Section 5.4.

4.8.4 Reward Representation

The reward provides feedback about the usefulness of the selected fault location. The primary signal is derived from the degradation produced by the injected fault.

If:

A_{clean} = clean-model accuracy

$A_{\text{fault}(c)}$ = accuracy after faulting channel c

then:

$$\Delta A(c) = A_{\text{clean}} - A_{\text{fault}(c)}$$

can be used as the principal fault-impact quantity.

A larger degradation indicates that the selected channel produced a greater effect on model performance and can therefore provide a stronger positive signal to the vulnerability-search process.

The reward formulation must remain fixed across TD3 experiments so that changes in search behaviour can be evaluated independently of changes in the evaluation metric.

4.8.5 TD3 Actor

The actor represents the policy used to select the next candidate fault location. Given the current state s_t , the actor produces an action:

$$a_t = \pi_{\theta}(s_t)$$

where θ represents the actor parameters.

The actor therefore learns a mapping between the current vulnerability-search state and a candidate fault location.

During exploration, controlled action variation can be introduced so that the agent does not repeatedly select only the currently known high-impact regions before sufficient exploration has occurred.

4.8.6 TD3 Critic

TD3 employs two critic networks to estimate the value associated with the selected action. The use of multiple critics is intended to reduce the effect of overestimated value predictions.

The critics estimate:

$$Q_{\phi 1}(s_t, a_t)$$

and

$$Q_{\phi 2}(s_t, a_t)$$

The smaller of the two estimates is used when constructing the target value:

$$y_t = r_t + \gamma \min[Q_{\phi 1}'(s_{(t+1)}, a'), Q_{\phi 2}'(s_{(t+1)}, a')]$$

where:

γ = discount factor

Q' = target critic network

a' = target action.

This double-critic structure is one of the defining mechanisms of TD3.

4.8.7 Delayed Policy Updates

TD3 delays actor updates relative to critic updates. The critics are therefore updated more frequently, while the actor and target networks are updated less frequently.

The purpose of this mechanism is to reduce instability caused by updating the policy using rapidly changing or inaccurate value estimates.

Within the proposed framework, delayed policy updates are maintained as part of the TD3 implementation while the fault-injection environment supplies the domain-specific reward signal.

4.8.8 Target Policy Smoothing

TD3 also incorporates target policy smoothing by adding controlled noise to the target action during critic target calculation.

Conceptually:

$$a' = \pi_{\theta'}(s_{(t+1)}) + \varepsilon$$

where ε represents clipped target-policy noise.

This mechanism reduces excessive dependence on narrow peaks in the learned value function and contributes to more stable critic learning.

4.8.9 Replay Buffer

The TD3 agent stores previously observed transitions in a replay buffer.

Each transition can be represented as:

$$(s_t, a_t, r_t, s_{(t+1)}, d_t)$$

where:

d_t indicates whether the current search episode has terminated.

During training, batches of previously observed transitions are sampled from the replay buffer to update the critic and actor networks.

The replay mechanism allows the agent to reuse fault-injection observations rather than relying exclusively on the most recent experiment.

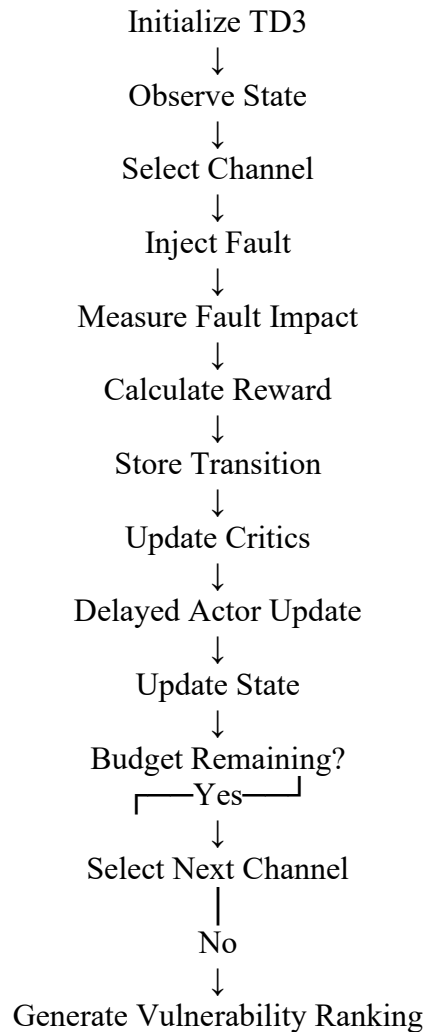
4.8.10 Vulnerability-Discovery Loop

The complete TD3 vulnerability-discovery procedure is:

1. Initialize the DNN and vulnerability-search environment.
2. Initialize the TD3 actor, twin critics, target networks, and replay buffer.
3. Observe the current search state.
4. Select a candidate channel using the TD3 policy.
5. Apply the predefined fault to the selected channel.
6. Execute inference using the faulted DNN.
7. Measure the resulting fault-induced degradation.
8. Convert the measured degradation into the reward.
9. Store the transition in the replay buffer.
10. Update the TD3 critics.
11. Perform the delayed actor and target-network updates according to the TD3 procedure.
12. Update the search state.

13. Repeat until the predefined fault-injection budget is reached.
14. Aggregate the observed channel-level vulnerability measurements.
15. Generate the final vulnerability ranking.

The process can be summarized as:



4.8.11 Comparison with Baseline Methods

TD3 is evaluated against the alternative vulnerability-selection approaches defined in the project:

- Random channel selection
- Activation-based Top-K ranking
- Gradient/Taylor-based ranking
- Layer-wise DDPG

The comparison focuses on the ability of each approach to identify high-impact channels under equivalent experimental constraints.

The primary comparison measures include:

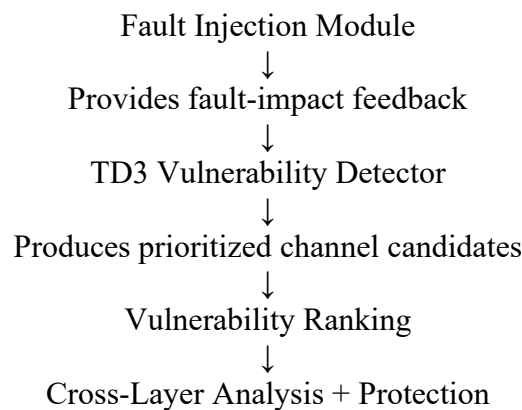
- Number of fault-injection evaluations
- Identified vulnerability
- Fault-induced accuracy degradation
- Ranking quality
- Computational overhead

The purpose is not to assume that TD3 will outperform every baseline, but to experimentally determine whether adaptive TD3-based selection provides an advantage under the defined fault model and evaluation budget.

4.8.12 Role of TD3 in the Overall Framework

TD3 is specifically responsible for the vulnerability-discovery stage. It does not directly perform fault injection, cross-layer interaction analysis, or model fine-tuning.

Its role can therefore be defined as:



This separation is important because it allows the contribution of the adaptive search strategy to be evaluated independently from the subsequent protection mechanism.

The output of this module is a set of evaluated channel vulnerabilities and a prioritized channel ranking. These outputs are passed to the Vulnerability Ranking and Selection module for further processing.

4.9 Vulnerability Ranking and Selection

The Vulnerability Ranking and Selection module converts the fault-impact observations obtained during vulnerability discovery into an ordered representation of channel-level vulnerability. Its purpose is to identify channels that exhibit comparatively high sensitivity to the defined fault condition and provide a consistent basis for subsequent cross-layer analysis and targeted protection.

The module receives observations generated by the Fault Injection Module and the TD3-based vulnerability-discovery process. Each observation associates a network layer, channel identifier, fault condition, and measured model degradation.

The basic input can be represented as:

$$O = \{(l_i, c_i, \Delta A_i)\}$$

where:

l_i = layer associated with channel i

c_i = channel identifier

ΔA_i = measured accuracy degradation caused by the injected fault

The primary vulnerability measure is:

$$\Delta A_i = A_{\text{clean}} - A_{\text{fault},i}$$

where:

A_{clean} = accuracy of the fault-free reference model

$A_{\text{fault},i}$ = accuracy obtained after injecting the defined fault into channel i

A larger value of ΔA_i indicates greater observed degradation under the evaluated fault condition.

4.9.1 Vulnerability Score

For the primary experimental ranking, the observed fault-induced degradation is used as the channel vulnerability score:

$$V_i = \Delta A_i$$

Therefore:

$$V_i > V_j$$

indicates that channel i produced greater measured degradation than channel j under the same experimental conditions.

Where the implementation uses a different normalized score, the transformation must be applied consistently to all compared channels and experimental methods.

The vulnerability score should be interpreted as an empirical sensitivity measure rather than an absolute physical reliability probability. It describes the observed effect of the defined software fault model on the evaluated DNN.

4.9.2 Channel Ranking

After collecting the available vulnerability observations, channels are ordered in descending order of vulnerability score:

$$V_{(1)} \geq V_{(2)} \geq \dots \geq V_{(N)}$$

where N represents the number of evaluated channels.

The resulting ranked list can be represented as:

$$R = [\\ (l_1, c_1, V_1), \\ (l_2, c_2, V_2),$$

$$(l_N, c_N, V_N)$$

The first entries in R represent the channels with the greatest measured fault-induced degradation.

This ranking is subsequently used to select channels for cross-layer interaction analysis and vulnerability-aware protection.

4.9.3 Top-K Vulnerable Channel Selection

A predefined number K of highly ranked channels can be selected from the complete ranking:

$$R_K = \text{TopK}(R)$$

The selected set contains the channels with the highest measured vulnerability scores among the evaluated candidates.

The value of K is determined according to the experimental objective. For cross-layer analysis, K must be sufficiently large to provide candidates from multiple layers while avoiding exhaustive combinations. For protection experiments, K is constrained by the predefined protection budget.

4.9.4 Layer-Aware Selection

Because the proposed framework investigates cross-layer fault interactions, the ranking retains the original layer identity of every channel.

For a selected channel:

$$r_i = (l_i, c_i, V_i)$$

The layer identifier l_i allows the framework to determine whether two selected channels originate from different layers.

For example, a candidate pair can be represented as:

$$P_{ij} = ((l_i, c_i), (l_j, c_j))$$

with:

$$l_i \neq l_j$$

Such pairs form the candidate set for simultaneous cross-layer fault analysis.

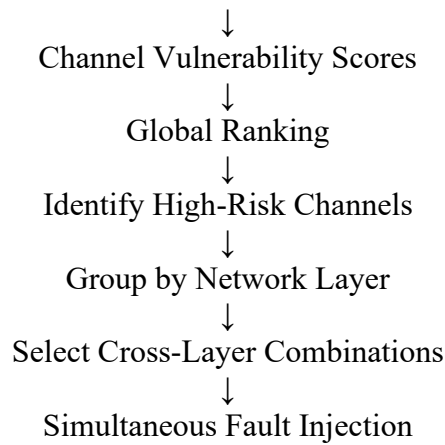
The ranking itself does not assume that channels from different layers will interact. It only provides the prioritized candidates from which cross-layer combinations can subsequently be evaluated experimentally.

4.9.5 Selection for Cross-Layer Interaction Analysis

The cross-layer analysis uses ranked vulnerable channels rather than randomly selecting arbitrary channel pairs.

The selection process is:

Fault-Injection Observations



For each selected pair, individual fault measurements are retained before the simultaneous experiment is performed.

This produces three principal observations:

D_i = degradation caused by channel i

D_j = degradation caused by channel j

D_{ij} = degradation caused by simultaneous faulting of i and j

These measurements are then passed to the Cross-Layer Interaction Analysis module.

4.9.6 Selection for Protection

The vulnerability ranking also determines which channels receive targeted fault-aware fine-tuning. Given a protection budget B , the framework selects a subset:

$$P_B \subseteq R$$

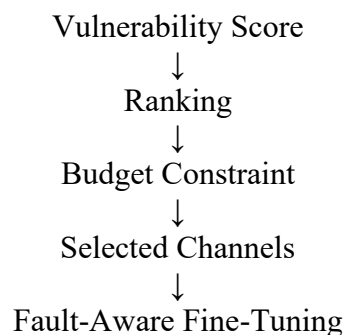
subject to the specified protection constraint.

The project evaluates protection budgets of:

- 1%
- 3%
- 5%
- 10%

The selected channels are then passed to the Vulnerability-Aware Fault Fine-Tuning module.

The resulting relationship is:



4.9.7 Baseline Ranking Strategies

The ranking and selection stage supports comparison with the baseline approaches defined for the project.

Random Selection

Channels are selected without using vulnerability information. This provides a reference for determining whether vulnerability-guided selection provides an advantage over uninformed selection.

Activation-Based Top-K

Channels are prioritized using activation-based information. The resulting selection is compared with fault-impact-based vulnerability ranking.

Gradient/Taylor-Based Ranking

Channels are prioritized using gradient or Taylor-based sensitivity information. This provides a sensitivity-based baseline that does not rely on direct fault-injection observations in the same manner as the proposed method.

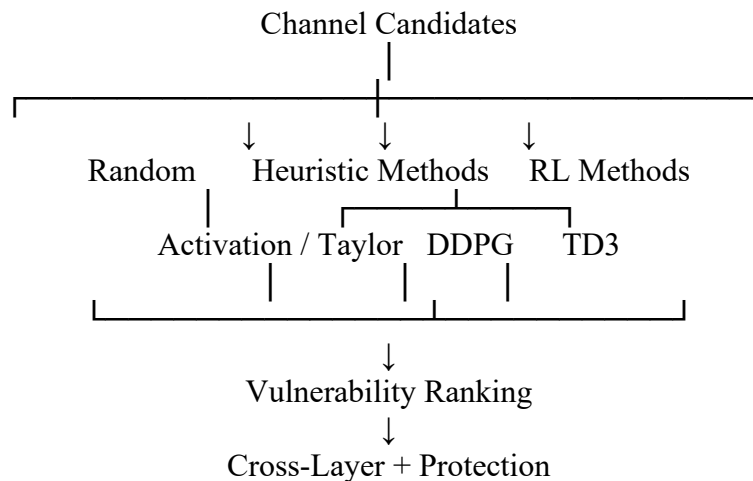
Layer-Wise DDPG

A layer-wise DDPG strategy provides the reinforcement-learning baseline. Its results are compared with TD3 under the same overall experimental objectives.

TD3-Based Selection

The TD3 approach uses adaptive fault-injection feedback to prioritize candidate channels during vulnerability discovery.

The comparison can therefore be represented as:



4.10 Cross-Layer Interaction Analysis

The Cross-Layer Interaction Analysis module investigates whether simultaneous faults affecting channels from different network layers produce degradation that differs from the degradation observed when the same channels are faulted individually. This module operates after the vulnerability-discovery stage and uses the resulting ranking to select candidate channels rather than evaluating every possible combination.

For a selected pair of highly ranked channels (l_i, c_i) and (l_j, c_j) with $l_i \neq l_j$, the module first retrieves or recomputes the individual degradation values D_i and D_j using the Fault Injection Module described in Section 4.6. It then injects both faults simultaneously during the same forward pass and records the combined degradation D_{ij} .

The comparison between D_{ij} and the individual measurements D_i and D_j provides evidence of interaction behaviour. Because DNN accuracy degradation is nonlinear and bounded, the "expected" combined effect against which D_{ij} is compared is not taken as the simple sum $D_i + D_j$; a precise, bounded reference for independent behaviour, together with the resulting interaction indicator, is defined in Section 6.2. Conceptually, however, if D_{ij} is approximately consistent with this independent-fault reference, the two channels can be considered to behave largely independently under the defined fault model. If D_{ij} is substantially larger, the combination may be exhibiting an amplifying interaction; if D_{ij} is smaller than expected, one fault may be partially masking the effect of the other.

Because exhaustive pairwise evaluation across all channels would be computationally prohibitive, the module restricts its candidate set to the top-ranked channels identified during vulnerability discovery, grouped by their originating layer so that only cross-layer combinations are considered. This keeps the analysis within the project's resource-constrained objective while still covering the channels most likely to reveal meaningful interaction effects.

The module records, for every evaluated combination, the layer and channel identifiers, the individual degradation values, the combined degradation value, and a derived interaction indicator. These records feed directly into the discussion of cross-layer effects presented in Sections 6.2 and 8.5, and provide the evidence base for answering RQ3.

4.11 Adversarial-Sensitivity Comparison

The Adversarial-Sensitivity Comparison module provides a supporting analysis that relates the channels identified as fault-sensitive to channels that are sensitive to adversarial input perturbations. This comparison is not a primary protection mechanism; it is included to characterize whether hardware-fault vulnerability and adversarial sensitivity expose overlapping or distinct internal structures within the evaluated DNN.

The module generates adversarial examples using the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD) against the clean reference model. Hardware-fault vulnerability, $\Delta A(c) = A_{\text{clean}} - A_{\text{fault}}(c)$, and adversarial sensitivity, a channel-level activation change, are measured in different units and are not directly comparable in their raw form. For each generated adversarial input, channel-level activation changes are therefore first converted into a normalized channel sensitivity score using the common representation defined in Section 7.8, placing both quantities on the same 0–1 scale before the two forms of sensitivity are compared or used for the four-way channel classification below.

Each evaluated channel is then assigned to one of four categories: fault-only vulnerable, where degradation is observed predominantly under simulated hardware faults; adversarial-only sensitive, where degradation is observed predominantly under adversarial inputs; dual vulnerable, where both perturbation types produce measurable degradation; and relatively robust, where neither perturbation type produces significant degradation within the evaluated conditions.

The output of this module is a channel-level sensitivity table that can be cross-referenced against the vulnerability ranking produced in Section 4.9. The resulting overlap statistics are reported in Section 8.6 and discussed in relation to RQ3 and the broader question of whether reliability-oriented and security-oriented internal weaknesses coincide.

4.12 Vulnerability-Aware Fault Fine-Tuning

The Vulnerability-Aware Fault Fine-Tuning module converts the vulnerability ranking into a concrete robustness-enhancement procedure. Rather than exposing every channel of the network

to simulated faults during additional training, the module selects only the channels identified as most fault-sensitive and restricts fault-aware exposure to this selected subset.

During fine-tuning, each training batch is processed twice: once under clean conditions and once with the defined channel stuck-at-zero fault applied to one or more of the selected vulnerable channels. The resulting clean-data loss and faulted-data loss are combined into a single training objective, described in detail in Section 6.5, so that the network is encouraged to retain its normal predictive accuracy while becoming more tolerant to the simulated fault condition.

The selection of channels for fine-tuning is governed by the protection budget described in Section 4.13. Only channels within the budget are exposed to simulated faults during training; all remaining channels are trained using only the clean-data objective for that batch, preserving the resource-constrained character of the proposed approach.

The module also supports comparison across different channel-selection strategies. Because the same fine-tuning procedure can be applied using channels selected by TD3, DDPG, activation-based ranking, gradient/Taylor-based ranking, or random selection, the resulting protected models can be directly compared to isolate the effect of the selection strategy from the effect of the fine-tuning procedure itself.

The output of this module is a protected DNN together with training logs recording the clean and faulted loss trajectories, the number of fine-tuning epochs, and the channels exposed to simulated faults during training. These outputs form the input to the Protected DNN Evaluation stage described in Section 4.5, Step 15.

4.13 Budget-Aware Protection Strategy

The Budget-Aware Protection Strategy formalizes the constraint that only a limited proportion of the network's channels can be selected for targeted fault-aware fine-tuning. This reflects the practical consideration that additional robustness training carries a computational and training-time cost that grows with the number of protected channels.

For a network containing N candidate channels and a protection budget B expressed as a percentage, the maximum number of channels that can be selected for protection is given by $K = \text{round}((B / 100) \times N)$, where B is expressed as a percentage (e.g., $B = 1$ for a 1% budget) and N is the number of candidate channels. The framework evaluates four protection budgets: 1%, 3%, 5%, and 10%, allowing the relationship between protection coverage and robustness improvement to be studied across an order of magnitude of selected channels.

For a given budget, the top- K channels from the vulnerability ranking R described in Section 4.9 are selected as the protected set P_B . The same budget is applied consistently across every channel-selection strategy under comparison, so that random selection, heuristic ranking, DDPG-guided selection, and TD3-guided selection are all evaluated using an identical number of protected channels at each budget level.

The budget-aware design allows the framework to examine two related questions: whether increasing the protection budget produces a monotonic improvement in faulted-model robustness, and whether a vulnerability-guided selection strategy can achieve a given level of robustness using a smaller budget than an uninformed or heuristic strategy. Both questions are addressed experimentally in Sections 7.9 and 8.7.

Because clean-data accuracy is monitored throughout fine-tuning, the budget-aware strategy also provides a basis for identifying the point at which further increases in the protection budget yield

diminishing robustness gains relative to their computational cost. This trade-off is discussed further in Section 9.4.

5. REINFORCEMENT LEARNING-BASED VULNERABILITY DETECTION

5.1 Reinforcement Learning Formulation

Chapter 4 introduced the TD3-based vulnerability-discovery module at the level of the overall framework. This chapter presents the reinforcement-learning formulation in full detail, defining the environment, state, action, reward, and agent architecture used to guide fault-injection experiments toward high-impact channels.

The vulnerability-discovery problem is formulated as a Markov Decision Process in which an agent interacts with a DNN fault-injection environment over a sequence of discrete steps. At each step, the agent observes the current search state, selects a candidate channel, receives a reward proportional to the observed fault impact, and transitions to an updated state that reflects the accumulated search history.

This formulation is deliberately kept close to standard reinforcement-learning practice so that TD3 can be compared fairly with the layer-wise DDPG baseline under an identical problem definition. Any performance difference between the two algorithms can therefore be attributed to their underlying update mechanisms rather than to differences in problem formulation.

5.2 Environment Design

The vulnerability-search environment wraps the trained DNN and the Fault Injection Module described in Section 4.6. An episode begins with the clean reference model and a fixed evaluation batch drawn from the validation split of the dataset described in Section 7.2. At each step of the episode, the environment receives a candidate channel from the agent, applies the defined stuck-at-zero fault, executes a forward pass, and computes the resulting degradation.

An episode terminates either when the predefined number of search steps per episode has been reached or when the allocated fault-injection budget for the current training run has been exhausted. Multiple episodes are executed sequentially during training, allowing the agent to refine its policy using experience accumulated across many candidate channels.

The environment maintains a record of channels that have already been evaluated during the current run so that repeated evaluation of the same channel can be identified. This record is exposed to the agent through the state representation described in Section 5.3, allowing the policy to learn to balance exploration of unevaluated channels against exploitation of channels already known to produce high degradation.

5.3 State Representation

The state provided to the TD3 agent combines three categories of information: static network information, dynamic search information, and accumulated vulnerability information. Static network information identifies the layer and channel indexing scheme used by the environment, allowing the agent's continuous output to be mapped onto a valid discrete channel location.

Dynamic search information reflects the current position within the search episode, including the number of steps already taken and the number of steps remaining within the current fault-injection budget. Accumulated vulnerability information summarizes the degradation values observed for previously evaluated channels, providing the agent with feedback about which regions of the search space have already produced high-impact results.

This composite state representation is held fixed across all TD3 and DDPG experiments reported in this project, so that differences in vulnerability-discovery performance can be attributed to the learning algorithm rather than to differences in the information available to the agent.

5.4 Action Space

TD3 is designed for continuous action spaces, whereas channel selection is inherently a discrete decision. The action space is therefore defined as a continuous vector $\mathbf{a}_t \in \mathbb{R}^d$, where d is the dimensionality of a fixed candidate-location embedding rather than the number of channels itself. Every eligible (layer, channel) pair is assigned a corresponding point in this same embedding space in advance, using a fixed indexing function shared across all channels considered eligible for fault injection. Given a produced action $\mathbf{a}_t$, the discrete channel selected for that step is the eligible (layer, channel) pair whose embedding is nearest to $\mathbf{a}_t$ under Euclidean distance, resolved by first identifying the target layer from a designated sub-range of $\mathbf{a}_t$ and then the channel index within that layer from the remaining components.

The continuous action produced by the actor network is first clipped to the valid range and then converted into a discrete channel index through this mapping function. Because the mapping is fixed throughout training and evaluation, small changes in the continuous action can correspond to different discrete channels, which is a standard consideration when applying continuous-control algorithms such as TD3 or DDPG to discrete selection problems.

This action-space design was selected to allow a direct comparison between TD3 and the layer-wise DDPG baseline, both of which are continuous-control algorithms, without requiring separate discrete-action variants of either method.

5.5 Reward Function

The reward signal is derived from the accuracy degradation $\Delta A(c) = A_{\text{clean}} - A_{\text{fault}}(c)$ produced by the selected channel, consistent with the vulnerability measure defined in Section 4.7. A channel that produces a larger accuracy degradation when faulted receives a proportionally larger reward, encouraging the agent to prioritize channels with greater observed impact.

To discourage repeated evaluation of the same channel without providing new information, the environment applies a small penalty when the agent selects a channel that has already been evaluated within the current search run. This encourages the agent to balance exploitation of known high-impact regions against exploration of unevaluated channels within the fault-injection budget.

The reward formulation is held constant across all TD3 experiments reported in this project. Any changes to the underlying degradation measure, such as substituting a loss-based measure for the accuracy-based measure, would require re-evaluating the reported results under the revised formulation.

5.6 TD3 Agent Architecture

The TD3 agent used in this project follows the standard actor-critic architecture described in Section 4.8, consisting of a single actor network, two independent critic networks, and corresponding target networks for the actor and both critics. The actor and critic networks are implemented as fully connected feed-forward networks that take the state representation described in Section 5.3 as input.

Training proceeds using the standard TD3 update procedure: the twin critics are updated at every training step using the minimum of their two target estimates, while the actor and all three target

networks are updated only once every fixed number of critic updates, consistent with the delayed policy-update mechanism described in Section 4.8.7. Target policy smoothing, described in Section 4.8.8, is applied by adding clipped noise to the target action during critic-target computation.

Transitions collected during vulnerability search are stored in a replay buffer and sampled in mini-batches for each update, allowing the agent to reuse fault-injection observations gathered across multiple episodes rather than relying solely on the most recent search step.

5.7 Vulnerability Search under Limited Injection Budget

The central operational constraint of the vulnerability-discovery stage is the fault-injection budget: the maximum number of fault-injection evaluations that the agent is permitted to perform before the search must terminate and a ranking must be produced. This budget reflects the practical requirement that vulnerability assessment should not require exhaustive evaluation of every channel in the network.

Under a limited budget, the agent must learn to identify high-impact channels using comparatively few evaluations, which places a premium on efficient exploration. The framework evaluates TD3 under several budget levels to determine how quickly the discovered vulnerability ranking approaches the quality obtainable from a much larger number of evaluations, and how this efficiency compares with the random, heuristic, and DDPG-based baselines described in Sections 4.9.7 and 7.6.

The number of fault-injection evaluations consumed by the search is recorded independently for each method, allowing the comparison to account for any differences in the computational cost of the search process itself, not only the quality of the resulting ranking.

5.8 Vulnerability Ranking Generation

Once the fault-injection budget allocated to a given search run has been exhausted, the accumulated observations are aggregated into the channel-level vulnerability ranking described in Section 4.9. Each evaluated channel is associated with its measured degradation value, and the complete set of evaluated channels is sorted in descending order of this value to produce the ranked list R .

Because the TD3 agent concentrates its evaluations on channels it has learned to associate with higher degradation, the resulting ranking is expected to contain proportionally more high-impact channels near the top of the list than a ranking produced from an equivalent number of randomly selected evaluations. This property is examined directly in Section 8.2.

The resulting ranking is passed to the Cross-Layer Interaction Analysis module described in Section 4.10 and to the Budget-Aware Protection Strategy described in Section 4.13, forming the bridge between the reinforcement-learning component of the framework and its downstream analysis and protection stages.

6. FAULT-AWARE PROTECTION AND CROSS-LAYER ANALYSIS

6.1 Single-Channel Fault Analysis

Single-channel fault analysis forms the baseline unit of evaluation throughout VulnShield-DNN. For each channel considered eligible for fault injection, the framework records the accuracy degradation $\Delta A(c)$ and, where relevant, the corresponding change in loss produced by the defined stuck-at-zero fault. These per-channel measurements are the raw observations from which the vulnerability ranking described in Section 4.9 is constructed.

Because single-channel analysis evaluates each channel in isolation, it provides a controlled reference point against which the combined effects examined in cross-layer analysis can be compared. Every cross-layer combination evaluated in Section 6.2 is therefore preceded by single-channel measurements for each of its constituent channels, ensuring that the individual and combined effects are measured under identical experimental conditions.

The single-channel results also support the comparative evaluation between TD3, DDPG, and the heuristic baselines, since all of these methods ultimately produce their vulnerability estimates through single-channel fault injection, differing only in how candidate channels are selected for evaluation.

6.2 Cross-Layer Fault Interaction

Building on the Cross-Layer Interaction Analysis module described in Section 4.10, this section defines the specific procedure used to select and evaluate cross-layer channel combinations within the experimental pipeline. Candidate pairs are drawn from the top-ranked channels produced by each vulnerability-discovery method, grouped by their originating layer so that only combinations spanning different layers are considered.

For each selected pair (l_i, c_i) and (l_j, c_j) , the framework computes the individual degradations D_i and D_j and the simultaneous degradation D_{ij} using the same evaluation batch for all three measurements. Because D_i and D_j are absolute accuracy degradations ($D_i = A_{\text{clean}} - A_{\text{fault},i}$) rather than fractions of a unit baseline, they are first normalized against the clean-model accuracy before being combined, so that the independence assumption is applied to the actual clean-accuracy scale rather than an implicit baseline of 1. The normalized degradations are defined as $d_i = D_i / A_{\text{clean}}$ and $d_j = D_j / A_{\text{clean}}$. Treating each fault as an independent relative reduction in the surviving correct-prediction rate, the normalized independent-fault reference is defined as $d_{\text{ref}} = 1 - (1 - d_i)(1 - d_j)$, which is bounded between the larger of d_i and d_j and their sum, and reduces to approximately $d_i + d_j$ only when both are small. The observed simultaneous degradation D_{ij} is normalized in the same way, $d_{ij} = D_{ij} / A_{\text{clean}}$, and the interaction indicator is derived on this normalized scale as $I_{ij} = d_{ij} - d_{\text{ref}}$, allowing each combination to be classified as approximately independent when I_{ij} is close to zero, amplifying when I_{ij} is substantially positive, or masking when I_{ij} is substantially negative.

The number of cross-layer combinations evaluated is bounded by a fixed budget, consistent with the resource-constrained motivation described in Section 3.6. This budget is applied identically regardless of which vulnerability-discovery method produced the candidate ranking, ensuring that the interaction analysis itself does not favour any particular channel-selection strategy.

6.3 Fault-Sensitive Channel Selection

Fault-sensitive channel selection is the process by which the vulnerability ranking is converted into a concrete set of channels for subsequent fine-tuning. Given a ranking R produced by any of the evaluated discovery methods and a protection budget B , the top- K entries of R are selected, where K is determined as described in Section 4.13.

Selection is performed independently for each vulnerability-discovery method under comparison, producing a distinct protected-channel set for random selection, activation-based ranking, gradient/Taylor-based ranking, layer-wise DDPG, and TD3-based search at each evaluated budget level. This design allows the downstream fine-tuning and evaluation stages to isolate the effect of the selection strategy from the effect of the fine-tuning procedure itself.

The selected channels, together with their layer identifiers, are passed to the Vulnerability-Aware Fine-Tuning procedure described in Section 6.4.

6.4 Vulnerability-Aware Fine-Tuning

Vulnerability-aware fine-tuning extends the pretrained clean model using additional training epochs during which the selected fault-sensitive channels are exposed to the defined stuck-at-zero fault. During each fine-tuning step, a training batch is evaluated twice: once without fault injection, producing the clean-data loss, and once with the fault applied to one of the selected channels, producing the faulted-data loss.

Channels are exposed to the fault in a round-robin or randomly sampled manner across fine-tuning steps so that, over the course of training, every selected channel receives comparable exposure to the fault condition. This is intended to avoid biasing the fine-tuning process toward only the highest-ranked channel within the selected set.

The fine-tuning procedure uses a reduced learning rate relative to the original training run, consistent with standard fine-tuning practice, and is applied for a fixed number of epochs that is held constant across all compared channel-selection strategies and protection budgets so that differences in the resulting robustness can be attributed to the selection strategy rather than to differences in training duration.

6.5 Combined Clean and Faulted Training Objective

The fine-tuning objective combines the clean-data classification loss L_{clean} and the faulted-data classification loss L_{fault} into a single weighted objective: $L_{\text{total}} = \alpha \cdot L_{\text{clean}} + (1 - \alpha) \cdot L_{\text{fault}}$, where $\alpha \in [0, 1]$ controls the relative emphasis placed on preserving clean-data performance versus improving tolerance to the simulated fault.

Selecting α close to 1 prioritizes clean-accuracy preservation but may limit the robustness gain achievable within a fixed number of fine-tuning epochs, whereas selecting α close to 0 prioritizes fault tolerance at greater risk of clean-accuracy degradation. The value of α used in the reported experiments is held fixed across all compared methods and budgets so that any observed differences in the clean-versus-faulted trade-off can be attributed to the channel-selection strategy rather than to differences in the training objective.

This combined objective directly operationalizes the requirement, introduced in Section 2.2 as Challenge 8, that robustness improvement should not be evaluated independently of its effect on normal model performance.

6.6 Protection Budget Formulation

The protection budget formulation described conceptually in Section 4.13 is applied consistently throughout the fine-tuning experiments. For each of the four evaluated budgets, 1%, 3%, 5%, and 10%, the number of protected channels $K = \text{round}((B / 100) \times N)$ is computed once for the target network architecture and held fixed across every channel-selection strategy compared at that budget.

This consistent budget application allows the resulting comparison in Section 8.7 to isolate the effect of channel-selection quality: at a fixed number of protected channels, any difference in the resulting faulted accuracy, clean accuracy, or robustness improvement can be attributed to which channels were selected rather than to how many channels were protected.

6.7 Protected DNN Generation

The output of the fine-tuning pipeline is a protected DNN corresponding to each combination of channel-selection strategy and protection budget. Each protected model is stored together with the original clean reference model, enabling direct before-and-after comparison under both clean and faulted evaluation conditions.

Protected models are evaluated using the metrics defined in Section 7.10, including faulted accuracy on the fault conditions used during fine-tuning, faulted accuracy on selected unseen channel faults not used during training, and clean-data accuracy. This multi-condition evaluation, described further in Section 4.5 Step 15, is intended to determine whether the observed robustness improvement generalizes beyond the specific fault conditions encountered during fine-tuning, addressing RQ5 and RQ7.

7. EXPERIMENTAL METHODOLOGY

7.1 Experimental Setup

The experimental evaluation of VulnShield-DNN is conducted entirely in software using standard deep-learning libraries, without requiring physical fault-injection hardware. All experiments are performed on trained convolutional neural networks using controlled channel-level fault injection, consistent with the software-level research scope defined in Section 4.2.

Each experimental run begins by loading a pretrained clean model and the evaluation dataset described in Section 7.2, followed by establishing the clean-model baseline described in Section 4.5, Step 2. Vulnerability discovery, cross-layer analysis, and vulnerability-aware fine-tuning are then performed according to the workflow described in Section 4.5, using the fault model, baseline methods, and evaluation metrics defined in the remaining subsections of this chapter.

All reported comparisons between vulnerability-discovery and protection strategies are performed under matched experimental conditions, including identical datasets, fault models, evaluation batches, and, where applicable, identical fault-injection or protection budgets.

7.2 Dataset

The primary dataset used for experimental evaluation is CIFAR-10, a widely used benchmark consisting of 60,000 32×32 colour images distributed across ten object classes, with a standard split of 50,000 training images and 10,000 test images. CIFAR-10 is selected because it is small enough to support the large number of repeated fault-injection and fine-tuning experiments required by the proposed framework, while still providing a non-trivial classification task for convolutional architectures such as ResNet-18 and VGG-16.

The training split is used for the original clean-model training and for the fault-aware fine-tuning stage, while a held-out portion of the test split is used as the fixed evaluation batch for fault-injection experiments, ensuring that vulnerability measurements are computed consistently across all compared methods. A separate portion of the test split is reserved for reporting final clean and faulted accuracy so that fine-tuning decisions are not influenced by the final reporting data.

7.3 DNN Architectures

The primary architecture used throughout the experimental evaluation is ResNet-18, a convolutional neural network characterized by residual connections that allow the training of moderately deep networks. VGG-16, a sequential convolutional architecture without residual connections, is used as an additional architecture to examine whether the observed vulnerability-discovery and protection behaviour is specific to a single network design or observed more generally across architectures with different structural properties.

Both architectures are trained to standard clean-accuracy levels on CIFAR-10 prior to any fault-injection experiments, and the resulting trained weights are retained as the clean reference models against which all faulted and protected-model results are compared. Fault injection and channel selection are applied to the convolutional channels within each architecture's feature-extraction layers, consistent with the channel-level scope defined in Section 2.6.

7.4 Fault Model

The fault model used throughout the experiments is the channel stuck-at-zero fault defined in Section 4.6, implemented using a PyTorch forward hook that temporarily replaces the output of a selected convolutional channel with zero for the duration of a single forward pass, leaving the underlying model parameters unchanged.

This fault model was selected because it provides a simple, reproducible, and architecture-independent representation of a channel becoming non-functional, without requiring assumptions about the specific physical mechanism responsible for the fault. As noted in Section 4.6, the stuck-at-zero condition represents a controlled software simulation rather than a direct reproduction of a particular hardware failure mechanism, and the reported results should be interpreted accordingly.

A limited weight bit-flip condition is additionally used as a secondary validation fault model, described further in Appendix A, to provide an indication of whether the vulnerability patterns observed under the primary fault model are also reflected under a different, parameter-level fault representation.

7.5 Fault Injection Procedure

The fault-injection procedure follows the sequence described in Section 4.6: a target channel is selected, the corresponding forward hook is registered, a forward pass is executed on the fixed evaluation batch, the resulting accuracy and loss are recorded, and the hook is removed before the next evaluation. This procedure is applied identically regardless of which vulnerability-discovery method selected the target channel, ensuring that all methods are evaluated using the same underlying fault-injection mechanism.

For cross-layer experiments, the procedure is extended to register forward hooks on two or more channels simultaneously before executing the forward pass, allowing the combined effect of multiple faults to be measured within a single evaluation, consistent with the simultaneous-fault process described in Section 4.6.

7.6 Baseline Vulnerability Detection Methods

Five vulnerability-discovery methods are evaluated under the same experimental conditions. Random selection chooses candidate channels uniformly at random, providing a lower-bound reference for vulnerability-discovery efficiency. Activation-based Top-K ranking prioritizes channels according to their average activation magnitude under clean inputs. Gradient or Taylor-based ranking prioritizes channels according to a first-order sensitivity estimate derived from the gradient of the loss with respect to the channel output.

Layer-wise DDPG uses a single-critic actor-critic reinforcement-learning agent, structured at the granularity of network layers, to guide fault-injection selection, providing a reinforcement-learning baseline against which the proposed TD3-based method can be directly compared. TD3-based search, described in Chapter 5, is the primary proposed method and operates at the granularity of individual channels using the twin-critic, delayed-update formulation described in Section 4.8.

All five methods are evaluated using an identical fault-injection budget at each comparison point, and their resulting rankings are evaluated using the metrics defined in Section 7.10.

7.7 TD3 Experimental Configuration

The TD3 agent used in the reported experiments is configured with a fixed actor and critic network structure, a fixed discount factor, a fixed target-network update rate, and a fixed target-policy-smoothing noise level, consistent with standard TD3 practice. The delayed policy-update interval,

described in Section 4.8.7, determines how frequently the actor and target networks are updated relative to the twin critics.

The replay buffer capacity, mini-batch size, and number of training episodes are held fixed across all reported TD3 runs so that comparisons across different fault-injection budgets reflect differences in the amount of environment interaction available to the agent rather than differences in the underlying learning configuration. The same overall hyperparameter structure, adapted to the layer-wise action space, is used for the DDPG baseline to keep the two reinforcement-learning methods as comparable as possible.

7.8 Adversarial Testing using FGSM and PGD

Adversarial testing is performed using the Fast Gradient Sign Method (FGSM), a single-step gradient-based perturbation technique, and Projected Gradient Descent (PGD), an iterative extension of FGSM that applies repeated small perturbation steps within a bounded region around the original input. Both attacks are applied to the clean reference model using a fixed perturbation magnitude, consistent with standard practice for evaluating adversarial sensitivity.

For each channel c , the raw adversarial activation change $AC(c)$ is computed as the mean absolute difference between clean and adversarial activations for that channel, averaged over the evaluation batch. Because $AC(c)$ and the fault-injection degradation $\Delta A(c)$ are measured in different units, both are independently min-max normalized over the full set of evaluated channels to obtain a common channel-level sensitivity score in the range $[0, 1]$: $S_{\text{fault}}(c)$ and $S_{\text{adv}}(c)$ respectively. A channel is classified using fixed thresholds on these normalized scores, consistent with the four categories introduced in Section 4.11: fault-only vulnerable when $S_{\text{fault}}(c)$ is high and $S_{\text{adv}}(c)$ is low, adversarial-only sensitive when the reverse holds, dual vulnerable when both scores are high, and relatively robust when both scores are low. The resulting adversarial examples are used to measure channel-level activation changes, which are compared against the hardware-fault vulnerability ranking as described in Section 4.11. This testing is performed independently of the fault-injection experiments and does not influence the vulnerability ranking used for cross-layer analysis or protection.

7.9 Protection Budgets

Consistent with Sections 4.13 and 6.6, four protection budgets are evaluated: 1%, 3%, 5%, and 10% of the total number of candidate channels in the target architecture. For each budget, every vulnerability-discovery method described in Section 7.6 is used to select the corresponding number of channels, and the resulting fine-tuned models are evaluated and compared.

This design produces a matrix of experimental conditions spanning five channel-selection strategies and four protection budgets for each of the two evaluated architectures, allowing the framework to examine both the effect of channel-selection quality at a fixed budget and the effect of budget size for a fixed selection strategy.

7.10 Evaluation Metrics

The evaluation considers several complementary metrics. Vulnerability-discovery efficiency measures how quickly a given method identifies high-impact channels relative to the number of fault-injection evaluations consumed. Faulted accuracy measures classification accuracy under the defined fault condition, while clean accuracy measures classification accuracy without fault injection.

Robustness improvement is computed as the change in faulted accuracy between the unprotected clean model and the protected model under an equivalent fault condition. Computational overhead captures the additional time or number of forward passes required by a given vulnerability-discovery or protection strategy relative to the baseline random-selection strategy.

Where applicable, these metrics are reported separately for known fault conditions used during fine-tuning and for selected unseen channel faults, in order to distinguish genuine robustness improvement from overfitting to the specific fault conditions encountered during training.

7.11 Validation Strategy

To reduce the risk that reported results reflect a single favourable run, the fault-injection and fine-tuning experiments are repeated across multiple random seeds, and the resulting metrics are summarized using their central tendency and variability across runs. This applies both to the reinforcement-learning-based methods, whose training is inherently stochastic, and to the deterministic heuristic baselines, which are instead evaluated across multiple randomly sampled evaluation batches.

Cross-architecture validation, using ResNet-18 as the primary architecture and VGG-16 as an additional architecture, is used to assess whether the reported findings generalize beyond a single network design. Unseen-fault validation, described in Section 4.2, Principle 10, is used to assess whether protection generalizes beyond the specific channels and fault conditions used during fine-tuning.

8. EXPECTED RESULTS AND EVALUATION

This chapter describes the results anticipated from the experimental methodology defined in Chapter 7. Because the proposed framework has not yet been fully executed at the time of writing, the discussion below presents expected outcomes framed as hypotheses to be confirmed or revised once the corresponding experiments have been completed, consistent with the evidence-based positioning established in Section 3.10. Reported figures, where illustrative values are used, are placeholders indicating the expected direction and structure of the results rather than final experimental measurements.

8.1 Vulnerability Detection Performance

The vulnerability-discovery experiments are expected to confirm that fault sensitivity is non-uniformly distributed across the evaluated architectures, consistent with the literature reviewed in Chapter 3. A relatively small proportion of channels is expected to account for a disproportionately large share of the total observed accuracy degradation, while the majority of channels are expected to exhibit comparatively limited individual impact.

The experiments will determine whether TD3-based search identifies a higher concentration of high-impact channels within a fixed fault-injection budget than random selection, and whether its resulting ranking correlates more strongly with subsequently measured degradation than the heuristic activation-based and gradient/Taylor-based rankings, which rely on indirect rather than direct fault-impact information.

8.2 Search Efficiency under Limited Fault Budget

Vulnerability-discovery efficiency is expected to favour TD3 as the fault-injection budget becomes more constrained. Under very small budgets, adaptive search is expected to show the largest relative advantage over random selection, because it can concentrate evaluations on channels more likely to be impactful. As the budget increases toward a level approaching exhaustive evaluation, the advantage of adaptive search over random selection is expected to diminish, since random selection will eventually cover most of the channel space regardless of strategy.

The number of fault-injection evaluations required for TD3 to identify a given proportion of the total vulnerability mass is expected to be reported as the primary efficiency measure, allowing direct comparison against the other four discovery methods described in Section 7.6.

8.3 Comparison with Random and Heuristic Methods

The experiments will determine whether TD3-based search identifies high-impact channels more effectively than random selection under equal fault-injection budgets, testing the general premise that adaptive, feedback-driven search can outperform an uninformed baseline. The comparison against activation-based and gradient/Taylor-based ranking is expected to be less uniform: these heuristics may perform competitively on some layers or architectures, particularly where activation magnitude or gradient sensitivity correlates closely with fault impact, but are expected to be less reliable on layers where this correlation is weaker.

This comparison is intended to test, rather than assume, the premise established in Section 3.3 that fault-injection-based rankings can provide more directly grounded vulnerability information than indirect, normal-operation statistics.

8.4 Comparison with DDPG-Based Search

The experiments will determine whether TD3 demonstrates more stable training behaviour than the layer-wise DDPG baseline, reflected in lower variance in the vulnerability-discovery efficiency measured across repeated runs, testing the design motivation for TD3's twin-critic and delayed-update mechanisms described in Section 4.8. Because DDPG in this framework operates at layer granularity rather than channel granularity, TD3 is, by construction rather than by hypothesis, able to produce a more fine-grained ranking capable of distinguishing between individually vulnerable channels within the same layer.

The comparison is expected to show that any efficiency advantage of TD3 over DDPG becomes more pronounced under smaller fault-injection budgets, where the additional stability of TD3's updates has a proportionally larger effect on the quality of the resulting ranking.

8.5 Cross-Layer Fault Interaction Results

Consistent with the gap identified in Section 3.6, the cross-layer interaction experiments are expected to reveal that at least a subset of the evaluated channel combinations exhibit degradation that differs measurably from the degradation expected under an assumption of independence. Some combinations may show evidence of amplification, in which the observed simultaneous degradation exceeds the independently expected reference degradation defined in Section 6.2, while others may show evidence of masking, in which the observed degradation is lower than this independent reference.

The proportion of evaluated combinations showing non-trivial interaction, rather than the existence of interaction in any single combination, is expected to form the primary evidence used to address RQ3, and this proportion is expected to vary across network depth, with later-layer combinations potentially showing different interaction characteristics than earlier-layer combinations.

8.6 Fault versus Adversarial Sensitivity

The comparison between hardware-fault-sensitive and adversarially sensitive channels, described in Section 4.11, is expected to show partial rather than complete overlap between the two sets. A subset of channels is expected to fall into the dual-vulnerable category, exhibiting sensitivity under both simulated hardware faults and FGSM/PGD adversarial perturbations, while other channels are expected to be sensitive to only one of the two perturbation mechanisms.

This partial overlap, if confirmed, would support the position established in Section 3.7 that hardware-fault vulnerability and adversarial sensitivity are related but non-equivalent forms of internal weakness, reinforcing the treatment of the adversarial comparison as a supporting rather than primary analysis.

8.7 Protection Performance under Different Budgets

Faulted-accuracy recovery after vulnerability-guided fine-tuning is expected to increase with the protection budget, with the largest marginal improvement occurring between the 1% and 3% budget levels, where the smallest number of the most critical channels are first protected. Diminishing returns are expected at higher budgets, consistent with the non-uniform vulnerability distribution described in Section 8.1, since channels beyond the most vulnerable subset are expected to contribute progressively less to overall degradation.

At each budget level, the experiments will determine whether TD3-guided channel selection achieves greater robustness improvement than random or heuristic selection using the same

number of protected channels, testing whether a vulnerability-guided protected set more closely matches the channels that actually dominate observed degradation.

8.8 Robustness and Clean-Accuracy Trade-off

Clean-data accuracy is expected to decrease slightly as a result of vulnerability-aware fine-tuning, reflecting the joint training objective described in Section 6.5. This decrease is expected to remain small relative to the corresponding gain in faulted accuracy, particularly at lower protection budgets, where only a small fraction of the network's channels are exposed to simulated faults during training.

The experiments will determine whether the trade-off between clean-accuracy retention and faulted-accuracy improvement is more favourable for vulnerability-guided selection than for random or heuristic selection, testing whether protecting fewer, more consequential channels can achieve a comparable robustness gain while disturbing a smaller portion of the network's learned representation.

8.9 Computational Overhead

The computational overhead of vulnerability discovery is expected to be dominated by the number of fault-injection evaluations for the heuristic and random baselines, and additionally by the cost of agent training for TD3 and DDPG. TD3 is expected to require more computation per fault-injection evaluation than the heuristic baselines, owing to the cost of actor and critic updates, but is expected to require substantially fewer total fault-injection evaluations to reach a comparable ranking quality under constrained budgets.

For the protection stage, computational overhead is expected to scale approximately linearly with the protection budget, since fine-tuning cost is primarily determined by the number of channels exposed to simulated faults during training. The overall computational profile of the proposed framework is expected to be reported alongside its robustness outcomes so that efficiency and effectiveness can be assessed jointly rather than in isolation.

9. DISCUSSION

9.1 Interpretation of Vulnerability Patterns

If the expected results described in Section 8.1 are confirmed, the concentration of fault impact within a small subset of channels would reinforce the position, established throughout the literature review in Chapter 3, that DNN reliability analysis benefits from component-level rather than purely aggregate evaluation. Such a pattern would suggest that reliability-improvement resources are better allocated according to measured criticality than distributed uniformly across the network.

The specific layers or channel types in which vulnerability concentrates, if consistent across ResNet-18 and VGG-16, would provide practically useful guidance for future reliability engineering, indicating that certain structural positions within a convolutional network may warrant closer reliability scrutiny regardless of the particular training run or dataset.

9.2 Effectiveness of TD3-Based Search

An efficiency advantage for TD3 over random and heuristic selection, if observed, would support the central premise of Chapter 5: that channel-level vulnerability discovery benefits from an adaptive, feedback-driven search strategy rather than a fixed sampling or indirect-statistic-based ordering. A comparable or superior result relative to layer-wise DDPG would further support the specific choice of TD3, attributable to its twin-critic and delayed-update mechanisms, as a more stable reinforcement-learning formulation for this task.

Should TD3 fail to outperform the heuristic baselines on a particular architecture or layer, this would not necessarily undermine the broader framework, since the cross-layer analysis and protection stages depend on the quality of the resulting ranking rather than on the specific algorithm used to produce it. Such an outcome would instead motivate further investigation into which categories of layers benefit most from adaptive search.

9.3 Significance of Cross-Layer Interactions

Confirmation of measurable interaction effects among selected cross-layer channel combinations, as anticipated in Section 8.5, would directly address the gap identified in Section 3.6 and would demonstrate that channel-level vulnerability rankings, however accurate individually, are insufficient on their own to fully characterize DNN behaviour under multiple simultaneous faults.

The practical implication of such a finding would be that protection strategies relying solely on individually ranked channels may need to additionally account for combinations of channels whose simultaneous failure produces disproportionate degradation, an implication that would motivate future extensions of the framework toward combinatorial rather than purely individual protection strategies, as outlined in Section 10.3.

9.4 Effectiveness of Ranking-Guided Protection

If vulnerability-guided fine-tuning achieves greater robustness improvement than random or heuristic selection at equal protection budgets, as anticipated in Section 8.7, this would confirm that the value of a vulnerability ranking extends beyond analysis into a measurable downstream protection benefit, addressing RQ4 and RQ7 directly.

The observed diminishing returns at higher protection budgets, if confirmed, would suggest that practical deployments of the proposed approach could achieve a substantial proportion of the

achievable robustness improvement using a comparatively small protection budget, which would be a relevant consideration for resource-constrained deployment scenarios such as edge inference.

9.5 Limitations of the Proposed Framework

The proposed framework is subject to several limitations inherent to its software-based, resource-constrained design. First, the channel stuck-at-zero fault model, while practical and reproducible, represents only one possible abstraction of hardware-induced corruption and may not capture every characteristic of physical fault mechanisms such as bit-level corruption or transient timing errors.

Second, the cross-layer interaction analysis is restricted to selected high-ranked channel combinations rather than exhaustive pairwise or higher-order evaluation, meaning that interaction effects involving lower-ranked channels are not captured by the current design. Third, the experimental evaluation is limited to two convolutional architectures and a single benchmark dataset, and the extent to which the observed patterns generalize to substantially different architectures, tasks, or datasets remains an open question.

Fourth, the reinforcement-learning components introduce additional training cost and stochastic variability that must be accounted for when interpreting efficiency comparisons, and the reported results depend on the specific hyperparameter configuration described in Section 7.7 and Appendix B.

9.6 Threats to Validity

Internal validity is addressed through the use of a fixed fault model, fixed evaluation batches, and matched experimental conditions across all compared methods, as described in Chapter 7, reducing the risk that observed differences arise from inconsistent experimental setup rather than from genuine differences between methods.

Construct validity concerns the extent to which the defined metrics, particularly accuracy degradation as a vulnerability measure, adequately capture the concept of fault sensitivity intended by the research questions; alternative measures, such as loss-based degradation, may yield different rankings and are discussed as a direction for future validation.

External validity is limited by the use of CIFAR-10 and two specific architectures, and by the reliance on simulated rather than physical faults, as discussed in Section 9.5. Conclusion validity is addressed through repetition across multiple random seeds and evaluation batches, as described in Section 7.11, though the specific number of repetitions used constrains the statistical confidence that can be placed in small observed differences between closely performing methods.

10. EXPECTED CONCLUSION AND FUTURE WORK

10.1 Expected Conclusion

This project set out to investigate whether reinforcement-learning-guided channel identification, specifically using TD3, can provide a more efficient basis for DNN fault-vulnerability discovery than conventional random and heuristic ranking approaches, whether cross-layer fault interactions among vulnerable channels produce effects that cannot be inferred from individual channel measurements, and whether the resulting vulnerability information can be used to guide targeted, budget-constrained robustness enhancement.

Subject to the experimental results obtained through the methodology described in Chapter 7, the proposed VulnShield-DNN framework is expected to demonstrate that TD3-based search can identify high-impact channels more efficiently than random selection and at least competitively with heuristic and DDPG-based alternatives under constrained fault-injection budgets; that a measurable proportion of cross-layer channel combinations exhibit non-trivial interaction effects; and that vulnerability-guided fine-tuning under a defined protection budget can improve faulted-model robustness while retaining the majority of clean-data performance.

Taken together, these findings, if confirmed, would support the central thesis of this work: that connecting efficient channel-level vulnerability discovery, explicit cross-layer interaction analysis, and budget-aware protection within a single experimental framework provides a more complete basis for DNN reliability enhancement than treating these components in isolation, as identified through the research-gap analysis in Section 3.9.

10.2 Research Contributions

The principal contributions of this work, consistent with those introduced in Sections 1.6 and 3.10, are: a TD3-based formulation of channel-level fault-vulnerability discovery under a constrained fault-injection budget; an explicit experimental procedure for identifying and characterizing simultaneous cross-layer fault interactions among ranked vulnerable channels; a vulnerability-guided, budget-aware fault-aware fine-tuning pipeline that connects discovery directly to protection; and a comparative experimental framework evaluating TD3 against random, activation-based, gradient/Taylor-based, and DDPG-based alternatives under matched conditions.

A further contribution is the supporting comparison between hardware-fault vulnerability and adversarial sensitivity, which provides additional evidence about the relationship between these two forms of internal DNN weakness without conflating their underlying mechanisms, and the cross-architecture evaluation across ResNet-18 and VGG-16, which provides an initial indication of whether the observed patterns generalize beyond a single network design.

10.3 Future Extensions

Several directions extend naturally from the present work. The cross-layer interaction analysis could be extended from pairwise combinations toward higher-order combinations involving three or more simultaneously faulted channels, subject to the availability of additional computational budget, to determine whether higher-order interaction effects follow patterns similar to those observed for pairs.

The fault model itself could be extended beyond the channel stuck-at-zero condition to include additional representations such as bounded random perturbations, bit-flip faults at the parameter

level, or transient faults affecting only a subset of the batch, allowing the sensitivity of the reported findings to the specific fault model to be assessed directly.

The protection mechanism could be extended from fault-aware fine-tuning toward combining vulnerability-guided selection with complementary protection techniques such as selective redundancy or checksum-based detection, drawing on the protection strategies reviewed in Section 3.5, to examine whether vulnerability-guided selection improves the efficiency of these alternative mechanisms as well.

Finally, extending the experimental evaluation to additional architectures, including Vision Transformers, and to additional datasets beyond CIFAR-10 would help establish the generality of the proposed framework beyond convolutional networks trained on small-scale image-classification benchmarks, and validating the proposed software fault model against physical or RTL-level fault-injection experiments would help quantify the abstraction gap discussed in Section 3.2.

REFERENCES

- Ahmadilivani, M. H., Taheri, M., Raik, J., Daneshtalab, M., & Jenihhin, M. (2024). A systematic literature review on hardware reliability assessment methods for deep neural networks. *ACM Computing Surveys*, 56(6).
- Ahmadilivani, M. H., et al. (2024). Enhancing fault resilience of QNNs via ensemble techniques. *IEEE International Symposium on On-Line Testing and Robust System Design (IOLTS)*.
- Ahsaei, S., & Raji, M. (2025). LOST-ViT: Layer-wise sensitivity and criticality analysis for Vision Transformers under hardware faults. *Journal of Systems Architecture*.
- Burel, S., Evans, A., & Anghel, L. (2024). Feature-map-level checksum and consistency-based fault detection for semantic-segmentation networks. *Microelectronics Reliability*.
- Cavagnero, N., et al. Fault-aware training for hardware-reliable deep neural networks without inference-time overhead.
- Fernandes dos Santos, F., et al. Transient-fault-aware design and training for improving DNN reliability.
- Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 1587–1596.
- Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. *International Conference on Learning Representations (ICLR)*.
- Gutiérrez-Zaballa, J., et al. (2024). Layer-wise and bit-position fault injection analysis of embedded semantic-segmentation networks. *Journal of Systems Architecture*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
- Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Technical report, University of Toronto.
- Lillicrap, T. P., et al. (2016). Continuous control with deep reinforcement learning. *International Conference on Learning Representations (ICLR)*.
- Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. *International Conference on Learning Representations (ICLR)*.
- Mesa Mesa, J. D., & Pérez Holguín, W. J. (2025). Fault-injection analysis of hardware-implemented deep neural networks. *EIEI-ACOFI*.
- Reinforcement-learning-based bit-protection mask learning with layer-wise resiliency profiling and transfer learning across DNN architectures (2025). *Sensors*.
- Base work: Deep reinforcement-learning framework for identifying vulnerable neurons through guided fault injection (2026). *IEEE Transactions on Reliability*.
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *International Conference on Learning Representations (ICLR)*.
- Sun, Y., et al. (2025). Aphelios: Selective lock-step execution using criticality estimation for reliable DNN accelerators. *ACM Great Lakes Symposium on VLSI (GLSVLSI)*.
- Syed, R. T., et al. (2024). A reconfigurable CNN accelerator with selective redundancy for reliability-critical branches. *IEEE Access*.
- Taheri, M., et al. (2024). Activation-fault injection analysis in quantized systolic-array DNN accelerators. *Design, Automation & Test in Europe Conference (DATE)*.
- Tonetto, L., et al. Simultaneous fault injection and masking effects in deep neural networks.
- Yu, Y., Tan, C., & Huang, J. (2024). A survey and classification of fault-injection tools for deep neural networks according to fidelity, cost, and scope. *ACM Computing Surveys*.
- Distributed edge inference reliability study (2025). Criticality-aware protection for distributed CNN inference across edge and IoT devices. *Scientific Reports*.
- Systematic review of disruptive attacks against artificial neural networks (2025). Fault-injection security and protection mechanisms.

PyTorchFI / PyTorchALFI / TensorFI+ / enpheeph documentation. Software fault-injection frameworks for deep-learning models.

APPENDIX

A. Experimental Configuration

This appendix summarizes the software environment and secondary fault-model configuration used to support the experimental methodology described in Chapter 7. All experiments are implemented using PyTorch, with fault injection realized through forward hooks as described in Section 4.6, and with the TD3 and DDPG agents implemented as fully connected actor-critic networks as described in Sections 5.6 and 7.7.

The secondary weight bit-flip fault condition referenced in Section 7.4 is implemented by selecting a single parameter tensor element within a target channel and flipping a chosen bit of its floating-point representation, providing a parameter-level complement to the primary activation-level stuck-at-zero fault model.

B. Hyperparameter Settings

The reinforcement-learning components use a fixed discount factor, actor and critic learning rates, target-network soft-update coefficient, replay-buffer capacity, mini-batch size, delayed policy-update interval, and target-policy-smoothing noise scale, held constant across all reported TD3 and DDPG runs, as described in Section 7.7. The fault-aware fine-tuning stage uses a fixed number of epochs, a reduced learning rate relative to the original clean-model training, and the loss-weighting coefficient α described in Section 6.5, held constant across all compared channel-selection strategies and protection budgets.

The specific numerical values of these hyperparameters are to be finalized during implementation and reported alongside the corresponding experimental results, consistent with the reproducibility principle described in Section 4.2.

C. Additional Experimental Results

This section is reserved for supplementary results that support but are not central to the primary findings reported in Chapter 8, including per-layer vulnerability breakdowns, extended cross-layer combination results beyond those highlighted in Section 8.5, additional adversarial-sensitivity category statistics, and results obtained under the secondary weight bit-flip fault condition described in Appendix A. These results will be added following completion of the corresponding experiments.

